

Advanced Natural Language Processing & Large Language Models

Felix Neubürger

2026

Fachhochschule Südwestfalen, Ingenieurs- & Wirtschaftswissenschaften

Inhalte der Vorlesung

- Wie funktioniert Natural Language Processing
- Sprachdarstellung zum Rechnen
- Attentionmechanismus
- Transformerarchitektur
- von BERT zu DeepSeek-v3
- Wie es weitergehen kann
- Nutzungsmöglichkeiten: RAG, Agentensysteme
- NER, Hybride Pipelines, Wissensgraphen

Ziele der Vorlesung - Welche Fragen sollen beantwortet werden?

- Was sind die Grundlagen von Natural Language Processing (NLP)?
- Wie funktionieren Attention-Mechanismen und warum sind sie wichtig?
- Was ist die Transformer-Architektur und wie unterscheidet sie sich von anderen Ansätzen?
- Wie werden Sprachmodelle wie BERT und GPT trainiert und genutzt?
- Welche Herausforderungen und ethischen Fragen gibt es bei der Nutzung von LLMs?
- Welche praktischen Anwendungen und Zukunftsperspektiven gibt es für LLMs?



[<https://xkcd.com/2451/>]

Format der Vorlesung - Wie sollen diese Fragen beantwortet werden?

- Theroetischer Teil mit Folien
- Praktischer Teil in Gruppen an einem Projekt
- Gruppengröße 2 oder 3 Personen
- Einzelarbeit möglich wenn eigenes Thema vorhanden
- Abgabe der Ausarbeitung einen Tag vor der Veranstaltung in der Blockwoche
- Vorstellung der Projektergebnisse in der Blockwoche
- Gewichtung der Bewertung Projektausarbeitung (50%) und Vortrag (50%)



IN CS, IT CAN BE HARD TO EXPLAIN THE DIFFERENCE BETWEEN THE EASY AND THE VIRTUALLY IMPOSSIBLE.

<https://xkcd.com/1425/>



Wie funktioniert Natural Language Processing

- Definition und Ziele des NLP
- Herausforderungen bei der maschinellen Sprachverarbeitung
- Anwendungen von NLP in der Praxis



Definition und Ziele des NLP

- NLP steht für Natural Language Processing, die Verarbeitung natürlicher Sprache durch Computer.
- Ziel: Maschinen ermöglichen, menschliche Sprache zu verstehen, zu interpretieren und zu generieren.
- Anwendungen: Übersetzungen, Chatbots, Textanalyse, Sprachassistenten.



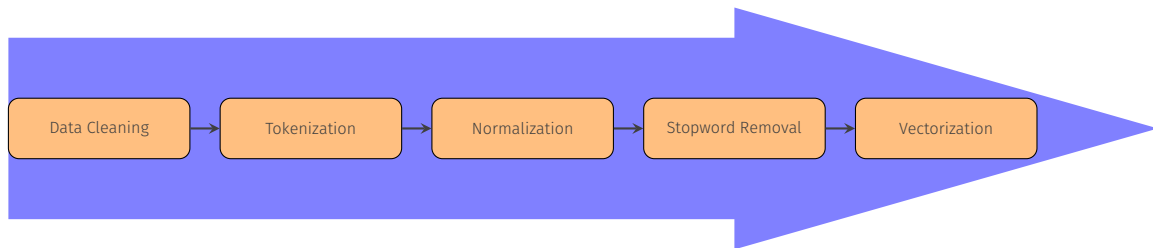
Herausforderungen bei der maschinellen Sprachverarbeitung

- Ambiguität: Mehrdeutigkeit in der Sprache.
- Kontextabhängigkeit: Bedeutung hängt vom Kontext ab.
- Umgang mit Synonymen und Homonymen.
- Verarbeitung großer Datenmengen und Rechenaufwand.

Anwendungen von NLP in der Praxis

- Sentiment-Analyse: Erkennung von Meinungen in Texten.
- Maschinelle Übersetzung: Automatische Übersetzung zwischen Sprachen.
- Sprachgesteuerte Assistenten: Siri, Alexa, Google Assistant.
- Textzusammenfassung: Automatische Erstellung von Textzusammenfassungen.

Text Preprocessing Pipeline



Data Cleaning

- **Definition:** Entfernen oder Korrigieren von fehlerhaften, unvollständigen oder irrelevanten Daten.

- **Schritte:**
 - Entfernen von Sonderzeichen, HTML-Tags und Emojis.
 - Korrektur von Rechtschreibfehlern.
 - Vereinheitlichung von Groß- und Kleinschreibung.

- **Ziel:** Verbesserung der Datenqualität für nachfolgende Verarbeitungsschritte.

Tokenization

- **Definition:** Zerlegung von Text in kleinere Einheiten (Tokens), z. B. Wörter oder Satzzeichen.
- **Arten:**
 - Wortbasierte Tokenization: "Das ist ein Satz." → ["Das", "ist", "ein", "Satz", "."]
 - Zeichenbasierte Tokenization: "Hallo" → ["H", "a", "l", "l", "o"]
 - Subwortbasierte Tokenization: "unbelievable" → ["un", "believ", "able"]
- **Herausforderungen:** Umgang mit zusammengesetzten Wörtern, Abkürzungen und Sonderzeichen.

Normalization

■ **Definition:** Vereinheitlichung von Textdaten, um Konsistenz zu gewährleisten.

■ **Schritte:**

- Umwandlung in Kleinbuchstaben: "Haus" → "haus".
- Entfernen von Akzenten: "café" → "cafe".
- Stemming: Reduktion auf Wortstamm, z. B. "running" → "run".
- Lemmatization: Rückführung auf Grundform, z. B. "better" → "good".

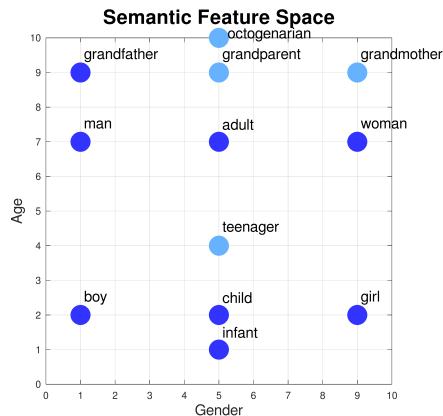
■ **Ziel:** Reduktion der Variabilität in den Daten.

Stopword Removal

- **Definition:** Entfernen von häufig vorkommenden Wörtern, die wenig Bedeutung tragen (z. B. "der", "und", "ist").
- **Vorgehen:**
 - Verwendung einer vordefinierten Stopword-Liste (z. B. "der", "die", "und", "ist", "ein", "zu").
 - Anpassung der Liste an den spezifischen Anwendungsfall.
- **Vorteile:**
 - Reduktion der Datenmenge.
 - Verbesserung der Modellleistung durch Fokus auf relevante Wörter.
- **Herausforderung:** Manche Stopwords können je nach Kontext wichtig sein.

Sprachdarstellung zum Rechnen

- Wortvektoren und Einbettungen (Embeddings)
- One-Hot-Encoding vs. verteilte Repräsentationen
- Word2Vec, GloVe und andere Einbettungsmethoden



Wortvektoren und Einbettungen (Embeddings)

- Ziel: Repräsentation von Wörtern in einem kontinuierlichen Vektorraum.
- Mathematische Definition:
 - Gegeben eine Menge von Wörtern $W = \{w_1, w_2, \dots, w_n\}$.
 - Eine Einbettung ist eine Funktion $f : W \rightarrow \mathbb{R}^d$, wobei d die Dimension des Vektorraums ist.
 - Beispiel: $f(w_i) = \mathbf{v}_i \in \mathbb{R}^d$.
- Vorteile:
 - Semantische Ähnlichkeit wird durch Nähe im Vektorraum dargestellt.
 - Reduktion der Dimensionalität im Vergleich zu One-Hot-Encoding.

One-Hot-Encoding vs. Verteilte Repräsentationen

■ One-Hot-Encoding:

- Jedes Wort wird als Vektor mit einer einzigen Eins und sonst Nullen dargestellt.
- Beispiel: Für $W = \{w_1, w_2, w_3\}$, w_2 wird als $[0, 1, 0]$ kodiert.
- Nachteile: Hohe Dimensionalität, keine semantische Information.

■ Verteilte Repräsentationen:

- Nutzen kontinuierliche Vektorräume, um semantische Beziehungen darzustellen¹.
- Ermöglichen die Nutzung von Modellen wie Word2Vec und GloVe².
- Wörter werden als dichte Vektoren in einem "niedrig"dimensionalen Raum dargestellt.
- Semantisch ähnliche Wörter haben ähnliche Vektoren.
- Beispiel: $f(w_1) = [0.2, 0.8]$, $f(w_2) = [0.3, 0.7]$.

¹Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv preprint arXiv:1301.3781.

²Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global Vectors for Word Representation. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP).

Word2Vec, GloVe und andere Einbettungsmethoden

■ Word2Vec:

- Skip-Gram-Modell: Vorhersage des Kontexts basierend auf einem Zielwort³.
- CBOW-Modell: Vorhersage des Zielworts basierend auf dem Kontext⁴.

■ GloVe (Global Vectors for Word Representation):

- Nutzt globale Wort-Kooccurenz-Matrizen⁵.
- Optimiert eine Zielfunktion, die Wortpaare und ihre Häufigkeiten berücksichtigt⁶.

■ Andere Methoden:

- FastText: Berücksichtigt Subwortinformationen.
- BERT-Embeddings: Kontextabhängige Einbettungen.

³Das Skip-Gram-Modell versucht, für ein gegebenes Zielwort die umgebenden Kontextwörter vorherzusagen.

⁴Das Continuous Bag of Words (CBOW)-Modell sagt ein Zielwort basierend auf den umgebenden Kontextwörtern vorher. Es ist effizienter als das Skip-Gram-Modell, aber weniger präzise bei seltenen Wörtern.

⁵GloVe basiert auf der Idee, dass die globale Häufigkeit von Wortpaaren in einem Korpus genutzt werden kann, um semantische Beziehungen zwischen Wörtern zu modellieren.

⁶Die Zielfunktion von GloVe minimiert den Unterschied zwischen der inneren Produktdarstellung von Wortvektoren und der logarithmierten Häufigkeit von Wortpaaren.

Neuartige Embeddings (Teil 1)

■ Kontextabhängige Embeddings:

- Modelle wie BERT⁷, GPT⁸ und T5⁹ generieren Embeddings, die den Kontext eines Wortes berücksichtigen.
- Beispiel: Das Wort "Bank" hat unterschiedliche Embeddings in den Sätzen "Ich sitze auf der Bank" und "Ich gehe zur Bank".

■ Sentence Embeddings:

- Repräsentieren ganze Sätze statt einzelner Wörter.
- Modelle wie Sentence-BERT (SBERT)¹⁰ ermöglichen semantische Suche und Textähnlichkeitsbewertung.

⁷Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. <https://arxiv.org/abs/1810.04805>

⁸Brown, T. et al. (2020). Language Models are Few-Shot Learners. <https://arxiv.org/abs/2005.14165>

⁹Raffel, C. et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. <https://arxiv.org/abs/1910.10683>

¹⁰Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. <https://arxiv.org/abs/1908.10084>

Neuartige Embeddings (Teil 2)

■ Multimodale Embeddings:

- Kombinieren Informationen aus verschiedenen Modalitäten wie Text, Bild und Audio.
- Beispiel: CLIP (Contrastive Language-Image Pretraining)¹¹ von OpenAI.

■ Graphbasierte Embeddings:

- Repräsentieren Wörter als Knoten in einem Graphen, wobei Kanten Beziehungen zwischen Wörtern darstellen.
- Beispiel: Node2Vec¹² und GraphSAGE¹³.

■ Adapter-basierte Embeddings:

- Ermöglichen die Anpassung vortrainierter Modelle an spezifische Aufgaben durch leichte Modifikationen.
- Reduzieren den Speicherbedarf im Vergleich zu vollständigem Fine-Tuning¹⁴.

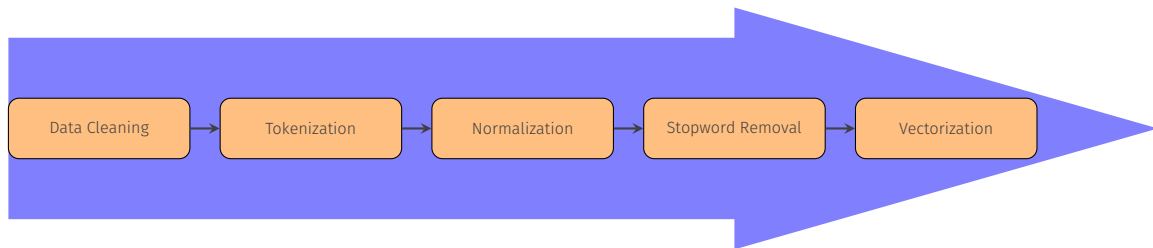
¹¹Radford, A. et al. (2021). Learning Transferable Visual Models From Natural Language Supervision. <https://arxiv.org/abs/2103.00020>

¹²Grover, A., & Leskovec, J. (2016). node2vec: Scalable Feature Learning for Networks. <https://arxiv.org/abs/1607.00653>

¹³Hamilton, W. et al. (2017). Inductive Representation Learning on Large Graphs. <https://arxiv.org/abs/1706.02216>

¹⁴Houlsby, N. et al. (2019). Parameter-Efficient Transfer Learning for NLP. <https://arxiv.org/abs/1902.00751>

Text Preprocessing Pipeline





Attention-Mechanismus

- Motivation für Attention in Sequenzmodellen
- Funktionsweise des Attention-Mechanismus
- Unterschied zwischen Self-Attention und Cross-Attention

Motivation für Attention in Sequenzmodellen

- Problem: In langen Sequenzen verlieren Modelle wie RNNs und LSTMs den Überblick über frühere Informationen.
- Lösung: Der Attention-Mechanismus ermöglicht es, gezielt auf relevante Teile der Eingabesequenz zu fokussieren.
- Beispiel: Bei der Übersetzung eines Satzes kann Attention bestimmen, welches Wort im Quelltext für ein bestimmtes Wort im Zieltext wichtig ist.

Funktionsweise des Attention-Mechanismus: Query, Key und Value

- **Eingabe:** Eine Sequenz von Eingabevektoren $X = \{x_1, x_2, \dots, x_n\}$, wobei $x_i \in \mathbb{R}^d$.
- **Transformation:** Jeder Eingabevektor wird durch trainierbare Gewichtungsmatrizen in Query (Q), Key (K) und Value (V) umgewandelt.
- **Berechnung:**

$$Q = XW_Q, \quad W_Q \in \mathbb{R}^{d \times d_k}$$

$$K = XW_K, \quad W_K \in \mathbb{R}^{d \times d_k}$$

$$V = XW_V, \quad W_V \in \mathbb{R}^{d \times d_v}$$

Hierbei sind W_Q , W_K und W_V trainierbare Gewichtungsmatrizen, und d_k , d_v sind die Dimensionen der Keys und Values.

- **Zweck:**
 - Q : Repräsentiert die Anfrage, welche Informationen benötigt werden.
 - K : Repräsentiert die Merkmale, die zur Beantwortung der Anfrage verwendet werden.
 - V : Repräsentiert die tatsächlichen Informationen, die weitergegeben werden.

Zusammenhang zwischen Query, Key und Value

■ Berechnung der Scores:

$$\text{Score}(Q, K) = \frac{QK}{\sqrt{d_k}}$$

Die Scores bestimmen, wie stark ein Query (Q) mit jedem Key (K) übereinstimmt.

■ Normalisierung der Scores:

$$\alpha_{ij} = \text{softmax}\left(\frac{q_i k_j}{\sqrt{d_k}}\right)$$

Die Softmax-Funktion wandelt die Scores in Wahrscheinlichkeiten um.

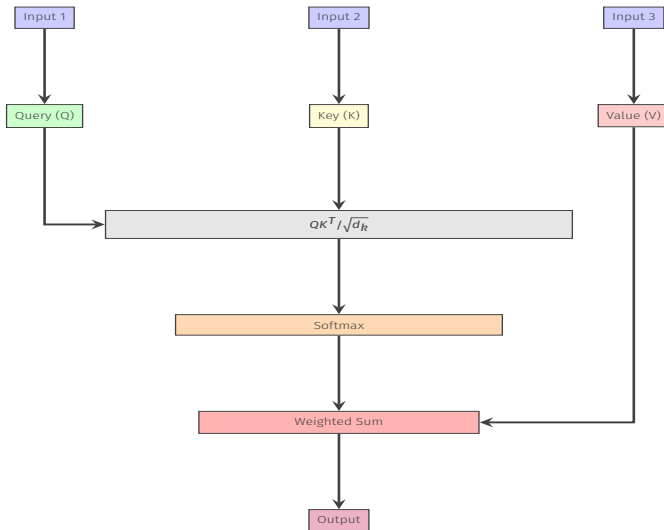
■ Gewichtete Summe der Values:

$$z_i = \sum_{j=1}^n \alpha_{ij} v_j$$

Die gewichtete Summe der Values (V) ergibt die Ausgabe des Attention-Mechanismus.

- **Interpretation:** Der Attention-Mechanismus ermöglicht es, relevante Informationen aus der Eingabesequenz basierend auf den Queries zu extrahieren.

Funktionsweise des Attention-Mechanismus



Self-Attention vs. Cross-Attention

Self-Attention:

- Jeder Token in der Sequenz bezieht sich auf alle anderen Tokens in derselben Sequenz.
- Beispiel: Kontextualisierung eines Wortes in einem Satz.

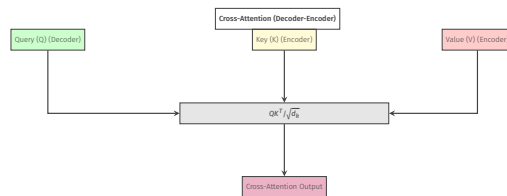
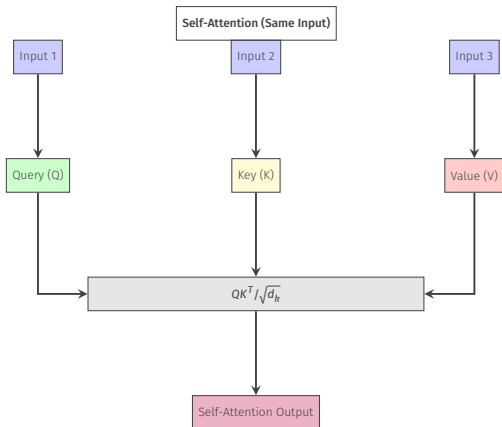
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V, \quad Q = K = V$$

Cross-Attention:

- Tokens in einer Sequenz beziehen sich auf Tokens in einer anderen Sequenz.
- Beispiel: Übersetzung, bei der der Zieltext auf den Quelltext achtet.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V, \quad Q \neq K = V$$

Self-Attention vs. Cross-Attention



Visualisierung der Attention-Matrix

- Die Attention-Matrix zeigt die Gewichte α_{ij} , die die Relevanz von Token j für Token i darstellen.
- Beispiel: Bei der Übersetzung eines Satzes zeigt die Matrix, welche Wörter im Quelltext für ein bestimmtes Wort im Zieltext wichtig sind.

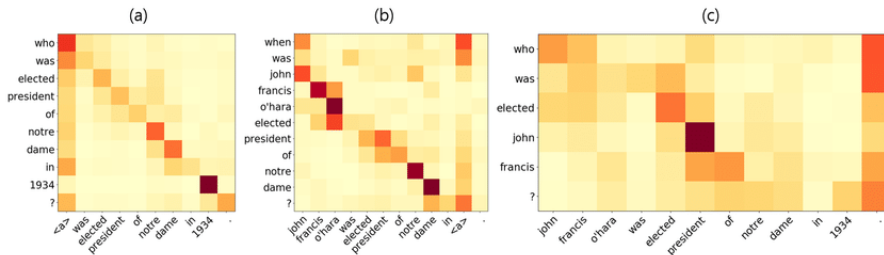


Abbildung: Beispiel einer Attention-Matrix. Kim, Yanghoon & Hwanhee, Lee & Shin, Joongbo & Jung, Kyomin. (2018). Improving Neural Question Generation using Answer Separation. 10.48550/arXiv.1809.02393.



Transformer-Architektur

- Überblick über die Transformer-Architektur
- Encoder-Decoder-Struktur
- Vorteile gegenüber rekurrenten Netzwerken

Überblick über die Transformer-Architektur

- Vorgestellt in "Attention Is All You Need" (Vaswani et al., 2017)¹⁵.
- Besteht aus zwei Hauptkomponenten:
 - **Encoder:** Verarbeitet die Eingabesequenz.
 - **Decoder:** Generiert die Ausgabesequenz.
- Verwendet Attention-Mechanismen und vollständig vernetzte Schichten.
- Vorteil: Parallelisierbarkeit im Vergleich zu RNNs.

¹⁵<https://doi.org/10.48550/arXiv.1706.03762>

Transformer-Architektur: Überblick

■ Encoder:

- Besteht aus mehreren Schichten.
- Jede Schicht enthält:
 - Multi-Head Self-Attention.
 - Feed-Forward-Netzwerk.

■ Decoder:

- Ähnlich wie der Encoder, aber mit zusätzlicher Maskierung.
- Enthält Cross-Attention, um Informationen vom Encoder zu nutzen.

■ Vorteile:

- Parallelisierbarkeit.
- Effektive Modellierung von Abhängigkeiten.

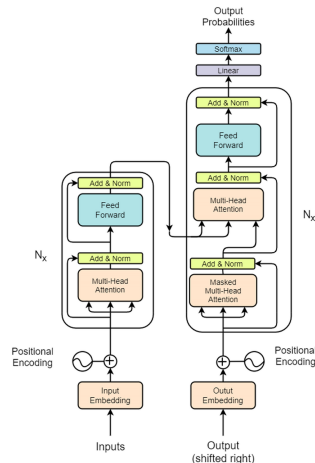


Abbildung: Transformer-Architektur^a

Encoder-Block: Multi-Head Attention

■ Multi-Head Attention:

- Berechnet die Attention über verschiedene Teile der Eingabesequenz.
- Formel:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}}\right)V$$

wobei:

- $Q = XW_Q$: Queries
- $K = XW_K$: Keys
- $V = XW_V$: Values
- W_Q, W_K, W_V : Gewichtungsmatrizen
- Multi-Head Mechanismus:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

wobei $\text{head}_i = \text{Attention}(QW_{Q_i}, KW_{K_i}, VW_{V_i})$.

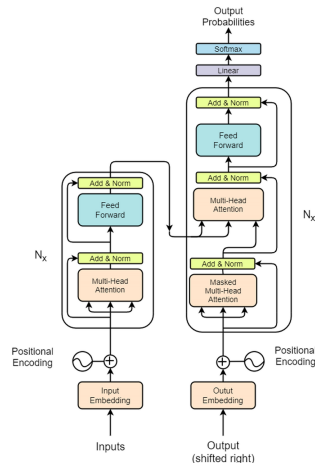


Abbildung: Transformer-Architektur^a

Encoder-Block: Feed-Forward-Netzwerk

■ Feed-Forward-Netzwerk:

■ Architektur:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

wobei:

- W_1, W_2 : Gewichtungsmatrizen
- b_1, b_2 : Bias-Vektoren
- ReLU: Aktivierungsfunktion
- Zweck: Transformation der Eingabe in einen höherdimensionalen Raum, um komplexere Muster zu lernen.

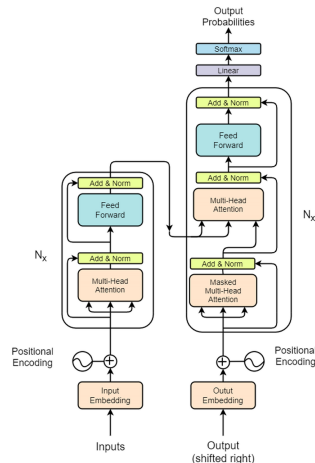


Abbildung: Transformer-Architektur^a

Encoder-Block: Add & Norm

■ Add & Norm:

- Residual Connection:

$$\text{Output} = \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x}))$$

wobei $\text{SubLayer}(\mathbf{x})$ entweder Multi-Head Attention oder das Feed-Forward-Netzwerk ist.

- Layer Normalization:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma} \cdot \gamma + \beta$$

wobei:

- μ : Mittelwert der Eingabe
- σ : Standardabweichung der Eingabe
- γ, β : Trainierbare Parameter
- Zweck: Stabilisierung des Trainings und Verbesserung der Konvergenz.

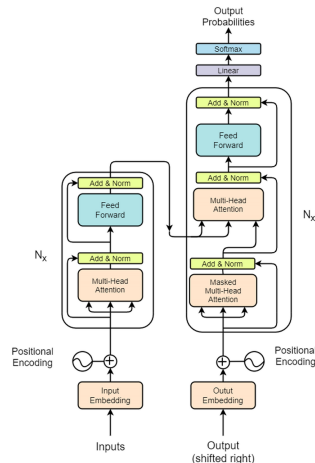


Abbildung: Transformer-Architektur^a

Decoder-Block: Multi-Head Attention

■ Masked Multi-Head Self-Attention:

- Verhindert, dass ein Token auf zukünftige Tokens zugreift.
- Maskierung der Attention-Matrix:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK}{\sqrt{d_k}} + M\right)V$$

wobei M eine Maske ist, die zukünftige Positionen ausschließt.

■ Cross-Attention:

- Verbindet den Decoder mit dem Encoder.
- Nutzt die Encoder-Ausgaben als Keys und Values.

■ Residual Connection and Layer Normalization:

- Wie im Encoder-Block, um Stabilität und Konvergenz zu verbessern.

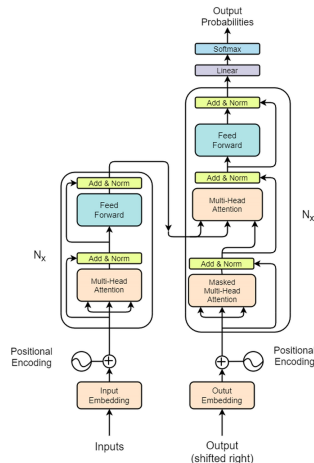


Abbildung: Decoder-Block^a

Decoder-Block: Feed-Forward-Netzwerk

■ Feed-Forward-Netzwerk:

■ Architektur:

$$\text{FFN}(x) = \text{ReLU}(xW_1 + b_1)W_2 + b_2$$

wobei:

- W_1, W_2 : Gewichtungsmatrizen
- b_1, b_2 : Bias-Vektoren
- ReLU: Aktivierungsfunktion
- Zweck: Transformation der Eingabe in einen höherdimensionalen Raum, um komplexere Muster zu lernen.
- **Residual Connection und Layer Normalization:**
 - Wie im Encoder-Block, um Stabilität und Konvergenz zu verbessern.

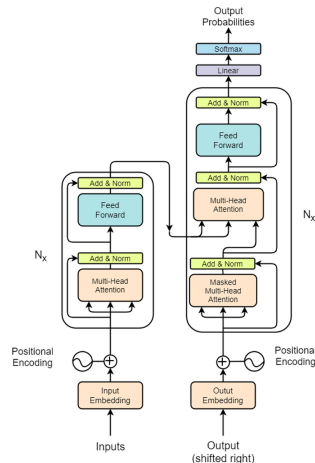


Abbildung: Decoder-Block^a

Decoder-Block: Add & Norm

■ Add & Norm:

- Residual Connection:

$$\text{Output} = \text{LayerNorm}(\mathbf{x} + \text{SubLayer}(\mathbf{x}))$$

wobei $\text{SubLayer}(\mathbf{x})$ entweder Masked Multi-Head Self-Attention, Cross-Attention oder das Feed-Forward-Netzwerk ist.

- Layer Normalization:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma} \cdot \gamma + \beta$$

wobei:

- μ : Mittelwert der Eingabe
- σ : Standardabweichung der Eingabe
- γ, β : Trainierbare Parameter
- Zweck: Stabilisierung des Trainings und Verbesserung der Konvergenz.

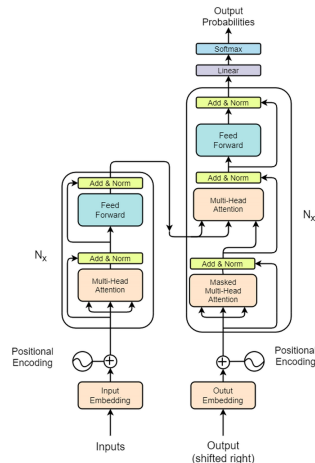


Abbildung: Decoder-Block^a

Vorteile der Transformer-Architektur

- **Parallelisierbarkeit:** Ermöglicht schnellere Trainingszeiten im Vergleich zu RNNs.
- **Langfristige Abhängigkeiten:** Kann Beziehungen zwischen weit entfernten Tokens modellieren.
- **Flexibilität:** Kann für verschiedene Aufgaben wie Übersetzung, Textklassifikation und mehr verwendet werden.
- **Skalierbarkeit:** Grundlage für große Sprachmodelle wie BERT und GPT.

Output-Möglichkeiten der Transformer-Architektur (Teil 1)

■ Sequenz-zu-Sequenz (Seq2Seq):

- Beispiel: Maschinelle Übersetzung (z. B. Englisch → Deutsch).
- Eingabe: Eine Sequenz von Tokens.
- Ausgabe: Eine Sequenz von Tokens in einer anderen Sprache.
- **Erreichung:** Verwendung eines Encoder-Decoder-Transformers, wobei der Encoder die Eingabesequenz verarbeitet und der Decoder die Ausgabesequenz generiert.

■ Sequenz-zu-Einzelwert (Seq2Single):

- Beispiel: Textklassifikation (z. B. Sentiment-Analyse).
- Eingabe: Eine Sequenz von Tokens.
- Ausgabe: Eine einzelne Klasse oder ein Wert.
- **Erreichung:** Hinzufügen einer Klassifikationsschicht (z. B. Softmax) am Ende des Encoders, um die Klasse vorherzusagen.

Output-Möglichkeiten der Transformer-Architektur (Teil 2)

■ Sequenz-zu-Token (Seq2Token):

- Beispiel: Fragebeantwortung (z. B. Auswahl eines Tokens als Antwort).
- Eingabe: Eine Sequenz von Tokens.
- Ausgabe: Ein spezifisches Token aus der Eingabe.
- **Erreichung:** Nutzung eines Modells wie BERT, das Start- und Endpositionen in der Eingabesequenz vorhersagt.

■ Sequenz-zu-Vektor (Seq2Vec):

- Beispiel: Satz- oder Dokumenteinbettung.
- Eingabe: Eine Sequenz von Tokens.
- Ausgabe: Ein Vektor, der die gesamte Sequenz repräsentiert.
- **Erreichung:** Extraktion des CLS-Tokens (bei BERT) oder Mittelung der Token-Embeddings, um die Sequenz zu repräsentieren.

Von BERT zu DeepSeek-v3

- Einführung in BERT und seine Architektur^a
- Weiterentwicklungen: GPT^b, RoBERTa^c, T5^d
- LLaMA^e, mistral^f, gemini^g, Claude^h
- Überblick über DeepSeek-v3 und seine Besonderheiten

^a<https://arxiv.org/abs/1810.04805>

^b<https://arxiv.org/abs/2005.14165>

^c<https://arxiv.org/abs/1907.11692>

^d<https://arxiv.org/abs/1910.10683>

^e<https://arxiv.org/abs/2302.13971>

^f<https://mistral.ai/>

^g<https://www.deepmind.com/>

^h<https://www.anthropic.com/>

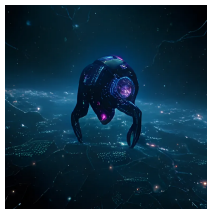
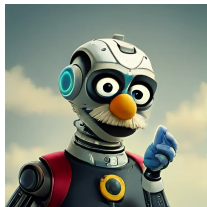


Abbildung: DeepSeek Janus Pro 7B-Interpretation von BERT und DeepSeek-v3 als KI-Modell.

BERT-Architektur: Überblick

■ BERT (Bidirectional Encoder Representations from Transformers):

- Entwickelt von Google AI (2018)¹⁶.
- Nutzt die Transformer-Encoder-Architektur.
- Bidirektionales Training: Betrachtet den Kontext von Wörtern sowohl links als auch rechts.

■ Ziele:

- Verbesserung der Sprachrepräsentation.
- Einsatz für verschiedene NLP-Aufgaben wie Fragebeantwortung, Sentiment-Analyse und mehr.

¹⁶<https://arxiv.org/abs/1810.04805>

BERT-Architektur: Aufbau

■ Eingabe:

- Tokenized Text: [CLS], Token 1, Token 2, ..., [SEP].
- Token-, Segment- und Positions-Embeddings.

■ Encoder:

- Mehrere Transformer-Encoder-Schichten.

■ Ausgabe:

- Kontextualisierte Token-Embeddings.
- CLS-Token für Klassifikationsaufgaben.

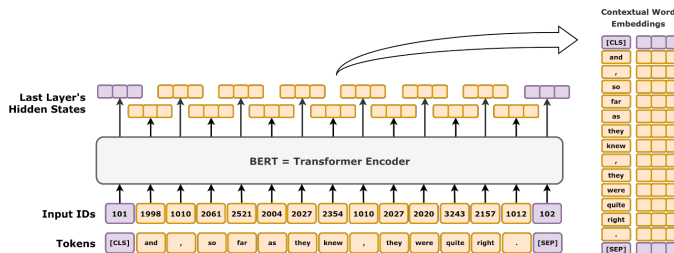


Abbildung: BERT-Architektur.

BERT-Architektur

BERT: Pretraining-Aufgaben

■ Masked Language Modeling (MLM):

- Zufälliges Maskieren von Tokens in der Eingabe.
- Ziel: Vorhersage der maskierten Tokens basierend auf dem Kontext.
- Beispiel: "Ich [MASK] ein Buch." → "Ich lese ein Buch."

■ Next Sentence Prediction (NSP):

- Ziel: Vorhersage, ob zwei Sätze aufeinander folgen.
- Beispiel:
 - Satz A: "Ich gehe einkaufen."
 - Satz B: "Danach koche ich Abendessen." → True

BERT: Fine-Tuning

■ Vorgehen:

- Pretrained BERT-Modell wird an spezifische Aufgaben angepasst.
- Hinzufügen einer zusätzlichen Schicht (z. B. Klassifikationslayer).

■ Beispiele für Aufgaben:

- Textklassifikation (z. B. Sentiment-Analyse).
- Fragebeantwortung (z. B. SQuAD).
- Named Entity Recognition (NER).

GPT-Architektur: Überblick

■ GPT (Generative Pre-trained Transformer):

- Entwickelt von OpenAI (2018)¹⁷.
- Nutzt die Transformer-Decoder-Architektur.
- Unidirektionales Training: Betrachtet nur den Kontext links vom aktuellen Token.

■ Ziele:

- Generierung von kohärentem und zusammenhängendem Text.
- Einsatz für Aufgaben wie Textgenerierung, Übersetzung und mehr.

¹⁷<https://arxiv.org/abs/2005.14165>

- Eingabe:

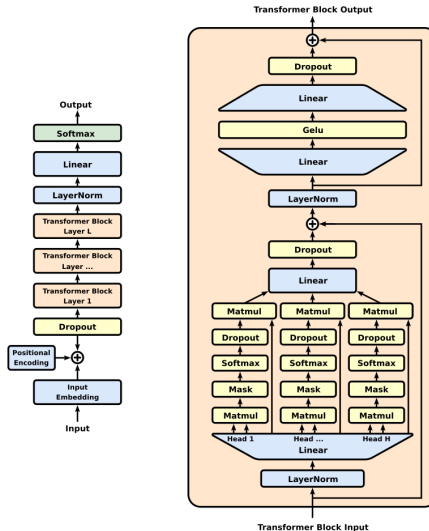
- Tokenized Text: [BOS], Token 1, Token 2, ..., [EOS].
- Token- und Positions-Embeddings.

- **Decoder:**

- Mehrere Transformer-Decoder-Schichten.
- Masked Multi-Head Self-Attention, um zukünftige Tokens auszuschließen.

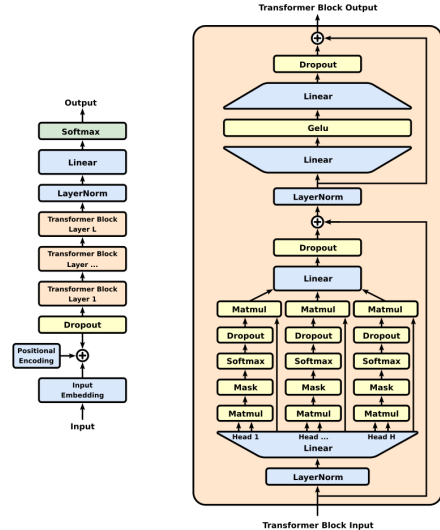
■ Ausgabe:

- Wahrscheinlichkeitsverteilung über das Vokabular für das nächste Token.



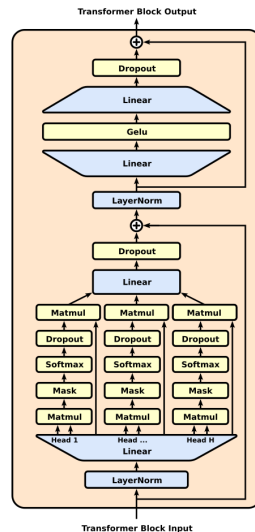
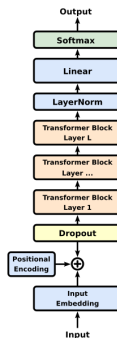
Überblick über die GPT-Architektur

- GPT (Generative Pretrained Transformer) basiert vollständig auf der Transformer-Decoder-Architektur.
- Die Architektur besteht aus einem Stapel identischer Transformer-Blöcke.
- Jeder Block enthält Self-Attention, Feedforward-Netze, Residual-Verbindungen und Layer Normalization.



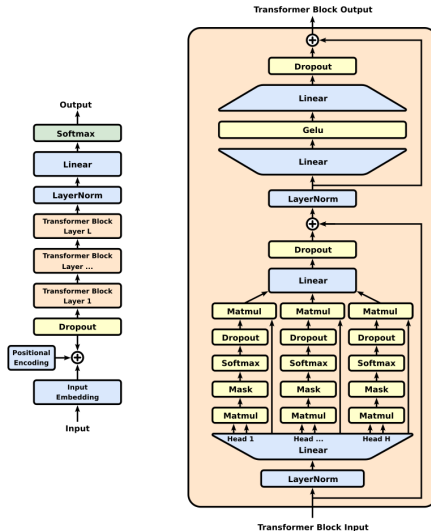
Input Embedding und Positionskodierung

- Der Input besteht aus Token-IDs, die in Vektor-Repräsentationen (Embeddings) umgewandelt werden.
- Positionsinformationen werden über **Positional Encoding** hinzugefügt, da das Modell keine Reihenfolge kennt.
- Die Summe aus Input Embedding und Positional Encoding geht in den ersten Transformer-Block.



Self-Attention Mechanismus

- Jeder Token schaut auf andere Tokens in seinem Kontext (bisherige Tokens).
- GPT nutzt **Masked Multi-Head Self-Attention**, um die Vorhersage zukünftiger Tokens zu verhindern.
- Besteht aus:
 - Lineare Transformationen zu Query, Key, Value
 - Maskierung der Zukunft
 - Softmax über Attention Scores
 - Gewichtete Summation der Werte



Feedforward-Netzwerk und Residual-Verbindungen

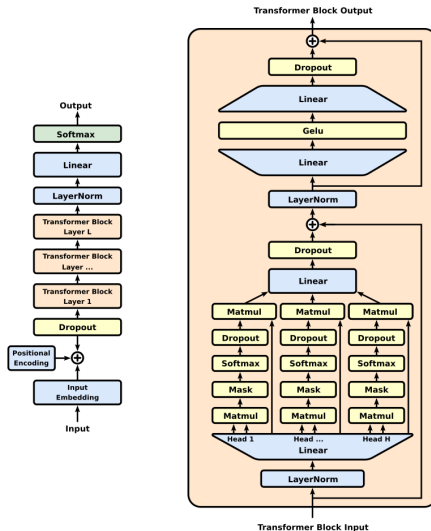
- Nach der Attention folgt ein Feedforward-Netz mit zwei linearen Schichten und einer Aktivierungsfunktion (GELU).
- GELU (Gaussian Error Linear Unit): Aktivierungsfunktion, definiert als:

$$\text{GELU}(x) = x \cdot \Phi(x)$$

wobei $\Phi(x)$ die kumulative Verteilungsfunktion der Standardnormalverteilung ist:

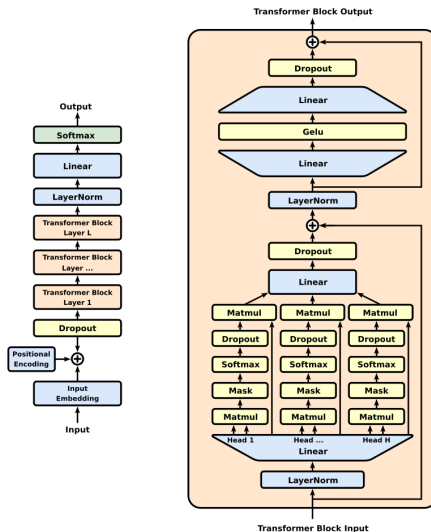
$$\Phi(x) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$$

- Vorteil: Glattere Approximation im Vergleich zu ReLU, wodurch das Training stabiler wird.
- Residual-Verbindungen sorgen für stabileres Training.
- LayerNorm** wird nach jeder Addition durchgeführt.
- Dropout** wird zur Regularisierung verwendet.



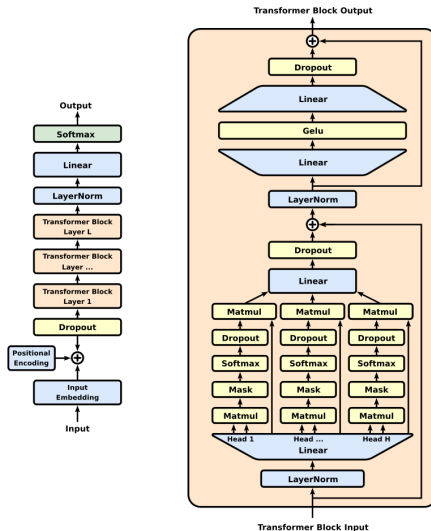
Mehrere Transformer-Layer

- Die Blöcke werden L-mal gestapelt (z. B. 12 für GPT-2 small, 96 für GPT-3).
- Jeder Layer hat dieselbe Architektur.
- Die Ausgabe des letzten Blocks wird verwendet, um Token-Vorhersagen durch ein lineares Projektionslayer + Softmax zu erzeugen.



Zusammenfassung

- GPT besteht aus reinen Decoder-Blöcken des Transformers.
- Nutzt Masked Self-Attention zur autoregressiven Textgenerierung.
- Durch Pretraining auf sehr großen Textmengen kann GPT Sprachverständnis und -generierung erlernen.



Pretraining: Autoregressives Training

- GPT wird durch **Autoregressive Language Modeling** trainiert:

$$P(x_1, x_2, \dots, x_n) = \prod_{t=1}^n P(x_t \mid x_{<t})$$

- Ziel: Nächstes Token vorhersagen basierend auf bisherigen Tokens.
- Loss-Funktion: Kreuzentropie-Loss zwischen vorhergesagtem und wahrem nächsten Token.
- Kontext wird vollständig nach links maskiert.

Trainingsdaten und Vorgehen

- GPT wird auf großen Korpora wie Common Crawl, Wikipedia, Bücher, Webseiten etc. trainiert.
- Tokenisierung erfolgt meist mit Byte-Pair Encoding (BPE).
- Typische Trainingsparameter (für GPT-2):
 - Kontextlänge: 1024 Tokens
 - Batchgröße: mehrere Millionen Tokens
 - Optimierung mit AdamW + Learning Rate Schedules
- Kein spezieller Pretraining-Task wie bei BERT (z.B. Masked LM oder NSP).

Fine-Tuning von GPT

- Nach dem Pretraining kann GPT für spezifische Aufgaben feinjustiert werden:
 - Textklassifikation
 - Fragebeantwortung
 - Dialogsysteme
 - Textgenerierung (z. B. ChatGPT)
- Fine-Tuning erfolgt meist mit task-spezifischen Daten.
- Architektur bleibt gleich – es werden oft nur letzte Layer angepasst.

GPT vs. BERT – Fundamentale Unterschiede

GPT (Decoder-basiert)

- Autoregressives Training (Zukunft maskiert)
- Nur Self-Attention nach links
- Einsatz: Textgenerierung, Dialogsysteme
- Output wird schrittweise erzeugt

BERT (Encoder-basiert)

- Masked Language Modeling (MLM)
- Bidirektionale Attention
- Einsatz: Klassifikation, Entitätenerkennung
- Kein autoregressiver Output

GPT erzeugt Sprache – BERT versteht sie.

Was ist Fine-Tuning?

- **Definition:** Anpassung eines vortrainierten Modells an eine spezifische Aufgabe oder Domäne.
- **Ziel:** Verbesserung der Leistung auf spezifischen Aufgaben durch zusätzliche Trainingsdaten.
- **Vorgehen:**
 - Start mit einem vortrainierten Modell (z. B. BERT, GPT).
 - Hinzufügen einer spezifischen Schicht (z. B. Klassifikationslayer).
 - Training auf einem domänenspezifischen Datensatz.

Vorteile des Fine-Tunings

- **Effizienz:** Reduziert den Bedarf an großen Trainingsressourcen, da das Modell bereits vortrainiert ist.
- **Anpassungsfähigkeit:** Ermöglicht die Anpassung an spezifische Aufgaben oder Domänen.
- **Verbesserte Leistung:** Höhere Genauigkeit und Relevanz für spezifische Anwendungen.
- **Wiederverwendbarkeit:** Vortrainierte Modelle können für verschiedene Aufgaben wiederverwendet werden.

Nachteile des Fine-Tunings

- **Overfitting:** Risiko, dass das Modell zu stark an den spezifischen Datensatz angepasst wird.
- **Datenbedarf:** Erfordert qualitativ hochwertige und ausreichend große Datensätze.
- **Rechenaufwand:** Kann trotz Vortraining immer noch ressourcenintensiv sein.
- **Komplexität:** Erfordert Fachwissen für die richtige Konfiguration und Optimierung.

Fine-Tuning-Techniken für LLMs

■ Parameter-Efficient Fine-Tuning (PEFT):

- Ziel: Reduktion der Anzahl der zu trainierenden Parameter.
- Vorteile: Geringerer Speicherbedarf und schnellere Trainingszeiten.

■ Low-Rank Adaptation (LoRA):

- Ziel: Effiziente Anpassung vortrainierter Modelle durch Low-Rank-Matrizen.
- Vorteile: Speicher- und Rechenaufwand werden drastisch reduziert.

Parameter-Efficient Fine-Tuning (PEFT)

■ Grundidee:

- Statt das gesamte Modell zu aktualisieren, werden nur wenige Parameter angepasst.
- Nutzung von Adapter-Schichten, Prompt-Tuning oder LoRA.

■ Mathematische Grundlage:

- Gegeben ein vortrainiertes Modell mit Parametern θ .
- PEFT optimiert nur einen kleinen Teil $\Delta\theta$, sodass:

$$\theta' = \theta + \Delta\theta$$

- Ziel: Minimierung des Verlusts L über $\Delta\theta$:

$$\min_{\Delta\theta} L(f(x; \theta + \Delta\theta), y)$$

■ Vorteile:

- Reduktion des Speicherbedarfs.
- Wiederverwendbarkeit des vortrainierten Modells.

Low-Rank Adaptation (LoRA): Grundidee

■ Motivation:

- Große Sprachmodelle haben Milliarden von Parametern.
- LoRA reduziert die Anzahl der zu trainierenden Parameter durch Low-Rank-Matrizen.

■ Ansatz:

- Zerlegung der Gewichtsmatrix W in zwei Low-Rank-Matrizen A und B :

$$W' = W + \Delta W, \quad \Delta W = AB$$

- $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{d \times r}$, wobei $r \ll d$.

■ Vorteile:

- Reduktion der Speicher- und Rechenkosten.
- Effizientes Fine-Tuning ohne Änderung der Hauptgewichte W .

Low-Rank Adaptation (LoRA): Mathematische Details

■ Modellanpassung:

- Gegeben eine Gewichtsmatrix W , wird die Anpassung ΔW durch:

$$\Delta W = AB$$

berechnet, wobei A und B trainierbar sind.

■ Optimierung:

- Ziel: Minimierung des Verlusts L über A und B :

$$\min_{A,B} L(f(x; W + AB), y)$$

■ Effizienz:

- Speicherbedarf: $O(r \cdot (d + d))$, wobei $r \ll d$.
- Rechenaufwand: Geringer als vollständiges Fine-Tuning.

Vergleich: PEFT, LoRA und Full Fine-Tuning

Eigenschaft	PEFT	LoRA	Full Fine-Tuning
Speicherbedarf	Gering	Sehr gering	Hoch
Rechenaufwand	Mittel	Niedrig	Sehr hoch
Flexibilität	Hoch	Mittel	Sehr hoch
Anwendungsfälle	Allgemein	Speziell für LLMs	Universell

Tabelle: Vergleich von PEFT, LoRA und Full Fine-Tuning.

Fine-Tuning Frameworks

■ Hugging Face Transformers:

- Umfangreiche Bibliothek für vortrainierte Modelle.
- Unterstützt einfache Anpassung und Fine-Tuning.
- <https://github.com/huggingface/transformers>

■ OpenAI Fine-Tuning API:

- Ermöglicht das Fine-Tuning von GPT-Modellen.
- Einfache Integration in bestehende Anwendungen.
- <https://platform.openai.com/docs/guides/fine-tuning>

■ PyTorch Lightning:

- Framework für vereinfachtes Training und Fine-Tuning.
- Unterstützt verteiltes Training und Mixed Precision.
- <https://www.pytorchlightning.ai/>

■ LoRA (Low-Rank Adaptation):

- Effizientes Fine-Tuning durch Reduktion der Parameteranzahl.
- Besonders geeignet für ressourcenbeschränkte Umgebungen.
- <https://github.com/microsoft/LoRA>

GPT-2: Verbesserungen gegenüber GPT

- **Größere Modelle:** GPT-2 wurde in verschiedenen Größen veröffentlicht (117M, 345M, 762M, 1.5B Parameter).
- **Training auf größeren Datenmengen:** GPT-2 wurde auf einem breiteren und vielfältigeren Korpus trainiert.
- **Verbesserte Textgenerierung:** GPT-2 erzeugt kohärentere und längere Texte.
- **Anwendungen:** Textzusammenfassung, Übersetzung, Dialogsysteme.

GPT-3: Skalierung und Few-Shot Learning

- **Skalierung:** GPT-3 hat 175 Milliarden Parameter, was es zu einem der größten Modelle macht.
- **Few-Shot Learning:** Kann Aufgaben mit wenigen Beispielen im Prompt lösen.
- **Anwendungen:** Codegenerierung, kreative Textgenerierung, komplexe Dialoge.
- **Herausforderungen:** Hoher Rechenaufwand, Bias in den generierten Texten.

GPT-4: Multimodalität und Verbesserungen

- **Multimodalität:** GPT-4 kann sowohl Text als auch Bilder als Eingabe verarbeiten.
- **Verbesserte Genauigkeit:** Bessere Leistung bei komplexen Aufgaben und längeren Kontexten.
- **Anwendungen:** Bildbeschreibung, multimodale Dialogsysteme.
- **Herausforderungen:** Noch höhere Anforderungen an Rechenressourcen.

GPT-4: Multimodale Verarbeitung

- **Multimodalität:** GPT-4 kann sowohl Text als auch Bilder als Eingabe verarbeiten.
- **Architektur:**
 - Erweiterung der Transformer-Architektur, um visuelle und textuelle Daten zu integrieren.
 - Gemeinsamer latent space für Text- und Bildrepräsentationen.
- **Anwendungen:**
 - Bildbeschreibung: Generierung von Texten basierend auf Bildern.
 - Visuelle Fragebeantwortung: Beantwortung von Fragen zu einem Bild.
 - Multimodale Dialogsysteme: Kombination von Text- und Bildinformationen in Konversationen.

GPT-4: Verarbeitung von Bildern und Text

■ Bildverarbeitung:

- Bilder werden durch ein visuelles Encoder-Modul (z. B. CNN oder Vision Transformer) in Features umgewandelt.
- Die Features werden in den Transformer integriert.

■ Textverarbeitung:

- Text wird wie in GPT-3 tokenisiert und in Embeddings umgewandelt.
- Gemeinsame Verarbeitung mit Bild-Features im Transformer.

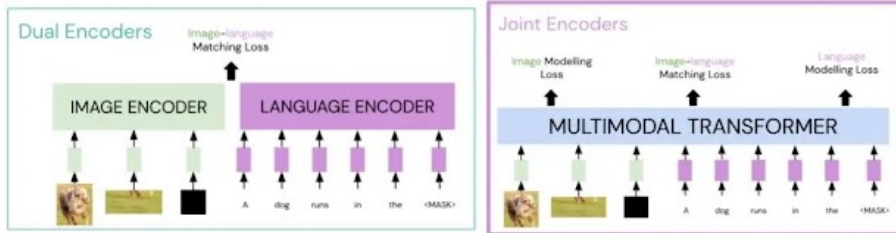


Abbildung: Multimodale Verarbeitung in Language models¹⁸

¹⁸https://lh3.googleusercontent.com/UGw_gUAQ0ja_28B-s-o0othiI2rmsIO2WJ_48s0xapPIfYJwK-pof8TgOqwnQwI0h4t6-RUw6saGjjWUDpqC224WwIPnnpiBfq5fLiKbsURczSwDGw=w616

GPT-4: Herausforderungen und Vorteile

■ Herausforderungen:

- Integration von Bild- und Textdaten in einem Modell.
- Hoher Rechenaufwand für Training und Inferenz.
- Bedarf an großen multimodalen Datensätzen.

■ Vorteile:

- Verbesserte Kontextverständnis durch Kombination von Text und Bild.
- Breitere Anwendungsbereiche, z. B. in der Medizin, Bildung und Unterhaltung.
- Fortschritt in multimodalen KI-Systemen.

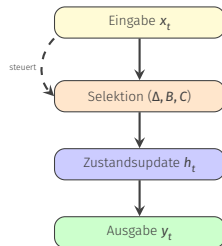
LLaMA: Open-Weight Modelle

- **LLaMA (Large Language Model Meta AI):** Entwickelt von Meta AI, mit Fokus auf Effizienz und Zugänglichkeit.
- **Open-Weight Modelle:** Verfügbar für die Forschungsgemeinschaft.
- **Größen:** Modelle mit 7B, 13B, 30B und 70B Parametern.
- **Anwendungen:** Forschung, Entwicklung von spezialisierten LLMs.

Mamba: Das Problem mit dem Transformer bei langen Sequenzen

- **Transformers skalieren quadratisch** in der Sequenzlänge – bei 1 M Token wird Attention unmöglich.
- Bisherige Alternative (RNNs, S4-SSMs): linear, aber time-invariant – können nicht kontextabhängig selektieren.
- **Kernidee Mamba^a**: SSM-Parameter werden eingabeabhängig – das Modell entscheidet pro Token, was es merkt oder vergisst.
- Ergebnis: lineare Skalierung + Transformer-Qualität, 5× höherer Inferenz-Durchsatz.

^aGu & Dao (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. <https://arxiv.org/abs/2312.00752>



Parameter sind Funktionen
der aktuellen Eingabe

Zustandsraum-Gleichungen (diskret):

$$y_t = C h_t$$

- **Klassisch (LTI):** A, B, C, Δ sind konstant – zeitinvariant, keine inhaltliche Selektion.
- **Mamba (Selektion):** B, C, Δ sind Funktionen von x_t – das Modell kann irrelevante Token vergessen und relevante merken.
- Δ steuert die Timescale: groß $\Rightarrow$ auf aktuellen Input fokussieren; klein $\Rightarrow$ Zustand beibehalten.

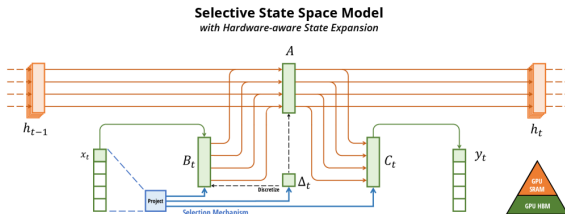


Abb.: SSM-Überblick – jeder Kanal hat eigenen latenten Zustand $\mathbf{h}$; selektive Parameter $\mathbf{A}, \mathbf{B}, \mathbf{C}$ sind eingabeabhängig (Gu & Dao, 2023, Fig. 1)

Mamba-Block und Hardware-aware Algorithmus

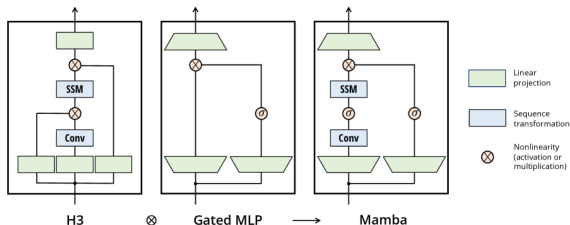


Abb.: Der Mamba-Block ersetzt den H3/MLP-Block durch einen einzigen, homogenen SSM-Block mit SiLU-Gate

(Gu & Dao, 2023, Fig. 3)

- **Kein Attention, kein MLP-Block** – ein einziger, wiederholt gestapelter Mamba-Block.
- **Eingabe:** Linear-Projektion + Erweiterungsfaktor $E = 2$.
- **Kern:** Selektiver Scan (S6) mit lokalem Faltungs-Prior.
- **Hardware-aware Scan:**
 - SSM-Parameter direkt vom HBM in schnelles SRAM laden.
 - Diskretisierung und Rekurrenz im SRAM – kein teures Materialisieren des Zustands.
 - Paralleler Scan statt sequenzieller Schleife: bis zu 3× schneller auf A100.
 - Rückwärtspfad via Recomputation (wie FlashAttention).
- **Trainingsmodus:** paralleler Scan; **Inferenzmodus:** reine Rekurrenz – konstante Zeit pro Token.

Mamba: Effizienz und Qualität im Vergleich

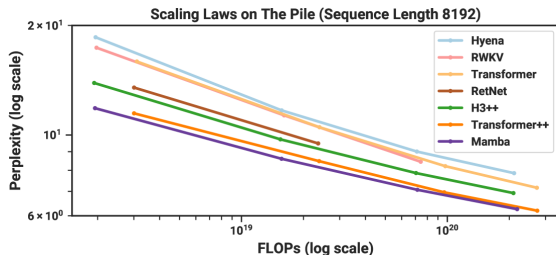


Abb.: Scaling Laws – Mamba als erstes attention-freies Modell, das Transformer++-Qualität erreicht (Gu & Dao, 2023, Fig. 6)

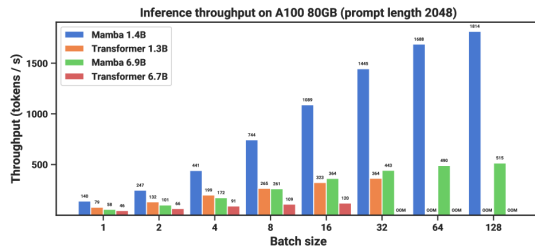
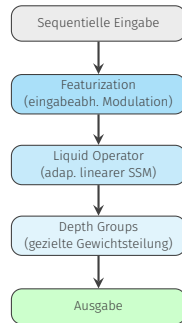


Abb.: Inferenz-Durchsatz – Mamba erreicht 5× höheren Throughput als Transformers gleicher Größe (Gu & Dao, 2023, Fig. 12)

- **Mamba-3B** übertrifft Transformer gleicher Größe und matcht Transformer doppelter Größe.
- Kein KV-Cache nötig $\Rightarrow$ deutlich höhere Batch-Größen möglich.
- Skaliert linear in der Sequenzlänge bis > 1 M Token.
- Anwendungen: Sprache, Genomik, Audio.

Liquid Foundation Models: Jenseits des Transformers

- **Liquid AI** (MIT-Ausgründung, 2023): Grundlagenmodelle aus ersten Prinzipien – kein GPT-Fork, keine Transformer-Ableitung.
- **Theoretische Wurzeln:**
 - Dynamische Systeme & ODE-basierte neuronale Netze
 - Strukturierte State-Space-Modelle (S4, Mamba)
 - Signalverarbeitung & numerische lineare Algebra
- **Kernidee:** Computational Units sind adaptive lineare Operatoren – ihre Parameter werden durch die Eingabe selbst moduliert (Featurization).
- **Bedeutung:** Erstmals übertrifft ein Nicht-Transformer-Modell Transformer gleicher Größe auf Standard-Benchmarks signifikant.^a



^aLiquid AI (2024). Liquid Foundation Models: Our First Series of Generative AI Models.
<https://www.liquid.ai/blog/liquid-foundation-models-our-first-series-of-generative-ai-models>

LFM: Mathematische Grundlagen – Liquid Time-Constant Netzwerke

- Ausgangspunkt: **Liquid Time-Constant (LTC)** neuronale Netze – kausal, kontinuierlich in der Zeit, ODE-basiert:¹⁹

$$\dot{x}(t) = -\left[\frac{1}{\tau} + f(x(t), I(t), t; \theta)\right] \odot x(t) + f(x(t), I(t), t; \theta)$$

- $x(t)$: Zustandsvektor, $I(t)$: Eingabe, τ : lernbare Zeitkonstante, $f(\cdot)$: neuronales Netz
- **“Liquid”**: Die effektive Zeitkonstante $\tau_{\text{eff}} = \tau / (1 + \tau \cdot f)$ hängt von der Eingabe ab – das System fließt adaptiv durch den Zustandsraum.
- **Vergleich Standard-RNN**: $\dot{x} = -x/\tau + f(x, I)$ – feste Zeitkonstante, keine Eingabeabhängigkeit der Dynamik.
- **Liquid-S4**: Kombination des LTC-Kerns mit der diagonalen + niedrig-rang (DPLR) Zustandsmatrix aus S4:

$$\dot{x}(t) = A(u_t) x(t) + B(u_t) u_t, \quad y(t) = C(u_t) x(t)$$

- Ergebnis: 87.32 % $\emptyset$ auf dem Long-Range-Arena-Benchmark – neuer State-of-the-Art (2022).

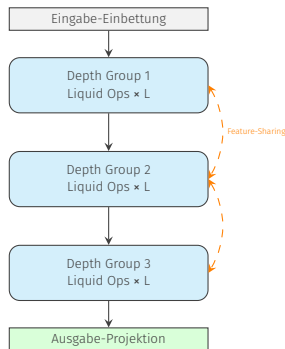
¹⁹Hasani et al. (2022). Liquid Structural State-Space Models. <https://arxiv.org/abs/2209.12951>

LFM-Architektur: Der neue Designraum adaptiver Operatoren

- LFMs vereinen und verallgemeinern bestehende Architekturen (Transformer, SSM, RNN) in einem **einheitlichen Designraum**:

Dimension	Funktion
Token-Mixing	Wie Positionen gemischt werden
Channel-Mixing	Wie Kanaldimensionen gemischt werden
Featurization	Eingabeabh. Modulation der Operatoren

- Depth Groups:** Gezielte Gewichtsteilung über Schichten – reduziert Parameter ohne Qualitätsverlust.
- Featurizer Interconnections:** Feature-Sharing zwischen Gruppen – ermöglicht globale Kontextintegration.
- Hardware-Anpassung:** Architekturen können automatisch für spezifische Hardware (Apple, Qualcomm, AMD, Cerebras) optimiert werden.



LFM-Modellreihe: LFM-1B, LFM-3B und LFM-40B (MoE)

■ LFM-1B (1,3B Parameter, dense)

- Höchste MMLU-Scores in der 1B-Klasse
- Erstmals übertrifft ein Nicht-Transformer signifikant Transformer gleicher Größe

■ LFM-3B (3,1B Parameter, dense)

- Übertrifft frühere 7B und 13B Transformer-Modelle
- 18,4 % kleiner als Phi-3.5-mini bei vergleichbarer Qualität
- Ideal für Edge-Deployment

■ LFM-40B (40,3B Parameter, MoE)

- Nur 12B aktive Parameter pro Forward-Pass
- Vergleichbar mit Modellen, die deutlich größer sind

■ Alle Modelle: 32k Kontextfenster; konstanter Inferenz-Speicher (kein wachsender KV-Cache)

MMLU (5-shot) – Ausgewählte Ergebnisse

Modell	Größe	MMLU
LFM-1B	1,3B	58,6
Llama-3.2	1B	45,5
Gemma-2	2B	52,2
LFM-3B	3,1B	66,2
Llama-3.2	3B	59,7
Phi-3.5-mini	3,8B	68,9
LFM-40B	40B	78,8
Mixtral-8x7B	47B	73,4

- Speicher: LFM-3B nutzt bei 32k Token weniger RAM als Llama-3.2-3B bei 4k Token.
- Kontext-Effizienz (RULER-Benchmark): LFM-3B 89,5 % bei 32k – besser als Llama-3.1-8B.

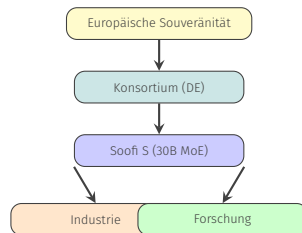
LFM vs. Transformer vs. Mamba: Architekturvergleich

Eigenschaft	Transformer	Mamba (SSM)	LFM
Attention	Quadratisch (n^2)	Keine	Keine
KV-Cache	Linear in n	Konstant	Konstant
Eingabeabh. Dynamik	Attention-Gewichte	$B, C, \Delta(x)$	Volle Featurization
Designraum	Fest (MHA+MLP)	SSM-Block	Unified (Token/Channel/Feat.)
Theoret. Grundlage	Seq.-zu-Seq.	Kontrolltheorie/SSM	ODE + SSM + Signalverarb.
Hardware-Optimierung	Allgemein	Flash-Convolution	Zielplattform-spezifisch
Long-Context Skalierung	Quadratisch	Linear	Sub-linear (konstanter Cache)
Open Source	Ja (viele)	Ja	Nein (proprietär)
Aktuelle Modellreihe	GPT-4, Llama, etc.	Mamba-1/2	LFM-1B/3B/40B, LFM2-Serie

- **LFM2 (2025):** Zweite Generation – Multimodalität (Audio, Vision), Sub-Milliarden Nanos für Edge, MoE-Varianten. Auf HuggingFace verfügbar: LiquidAI/LFM2.5-*.
- **Grenzen:** Proprietär (kein Open-Source), Code-Generierung und genaue Berechnungen noch schwächer als spezialisierte Modelle.

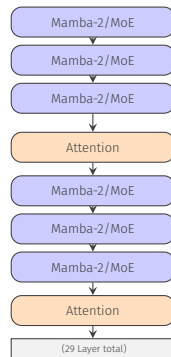
Was ist Soofi? – Souveräne KI Made in Germany

- **SOOFI** = Sovereign Open Source Foundation Models – ein deutsches Forschungskonsortium aus führenden Instituten, Universitäten und Startups.
- **Ziel:** Eine offene, europäische Alternative zu US- und chinesischen Frontier-Modellen (GPT-4o, Gemini, DeepSeek) – mit Fokus auf **Datensouveränität** und **industriellen Anwendungen**.
- **Infrastruktur:** Training vollständig auf der T-Systems Industrial AI Cloud (Deutsche Telekom) in Europa.
- **Konsortium:** KI-Bundesverband, Deutsche Telekom, Forschungseinrichtungen – gefördert vom Bundesministerium für Wirtschaft und Klimaschutz.
- **Status (2026):** Soofi S ist das erste veröffentlichte Modell – aktuell in der Preview-Phase für Industriepartner.



Soofi S: Architektur im Detail – Hybrides Mamba-2/MoE

- **Architekturtyp:** Hybrid Mamba-2 / Mixture-of-Experts (MoE) – inspiriert von Nvidias Nemotron-H.
- **Layer-Aufteilung:**
 - **23 Mamba-2/MoE-Layer:** effiziente lineare SSM-Schicht mit MoE-Routing.
 - **6 Attention-Layer:** für kontextabhängige, globale Interaktionen.
- **MoE-Details:**
 - 128 routing Experten + 1 shared Expert pro MoE-Layer.
 - **6 Experten aktiv pro Token** – sehr sparse Aktivierung.
- **Parameterzahl:** 30 B gesamt, nur **~3,5 B aktiv während Inferenz**.
- **Warum Mamba-2 + MoE?** Mamba liefert lineare Effizienz für lange Sequenzen; MoE skaliert Kapazität ohne proportional mehr Rechenzeit.



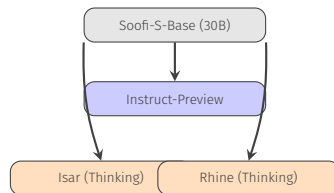
blau = Mamba/MoE orange = Attn

Soofi S: Pretraining-Daten – Woher stammt das Wissen?

- **Gesamtvolumen:** ~25 Billionen Token – vollständig von Grund auf trainiert (kein Basis-Modell übernommen).
- **Hauptsprachen:** Deutsch und Englisch (primär); Französisch, Italienisch, Spanisch (begrenzt). Englisch als Pivot-Sprache.
- **Wissens-Cutoff:** Ende 2025. Trainingsstart: April 2026.
- **Datenquellen (hochwertig, dokumentiert):**
 - **Web-Crawls:** Gefilterte Common-Crawl-Daten mit Qualitätsstufen.
 - **Bücher und Fachliteratur:** Deutschsprachige Textkorpora (Gutenberg, Wissenschaftsverlag-Digitalisate).
 - **Code:** GitHub-Repositories, Stack Overflow, Dokumentationen.
 - **Wissenschaftliche Artikel:** arXiv, PubMed, OpenAlex – mehrsprachig.
 - **Spezifisch Deutsch:** Nachrichtenarchive, Gesetzestexte, Verwaltungsdokumente, Normen (DIN/ISO).
- **Qualitätspipeline:** Sprachidentifikation → Deduplizierung → Qualitätsfilterung → Content-Filter.

Soofi S: Modellvarianten – Warum tragen sie Flussnamen?

- **Naming Convention:** Alle Fine-Tuning-Varianten sind nach deutschen Flüssen benannt – ein bewusstes Signal für nationale Identität und europäische Souveränität.
- **Varianten (Soofi S Beta):**
 - **Soofi-S-Base** (30B): Basis-Modell, kein Instruction-Tuning.
 - **Soofi-S-Instruct** (30B): Chat-optimiert, Instruction-Following.
 - **Soofi-S-Isar** (30B): Reasoning-Variante mit explizitem <think>-Block.
 - **Soofi-S-Rhine / Rhein** (30B): Zweite Reasoning-Variante mit CoT.
- **Warum zwei Reasoning-Varianten?** Parallele Exploration verschiedener RLVR-/Post-Training-Rezepte (analog DeepSeek-R1 vs. o1). Die beste Variante wird Basis der nächsten Generation.



Isar = Bayerischer Fluss
Rhine = Rhein (Westdeutschland)

Naming: Europäische Identität
und Datensouveränität

Soofi S: Einordnung und Bedeutung für Deutschland

- **Frontier-Modell** = Modell an der technologischen Leistungsgrenze – Soofi S ist das erste deutsche Modell, das diesen Anspruch erhebt.
- **Alleinstellungsmerkmale gegenüber GPT-4o/Gemini/DeepSeek:**
 - Vollständig **transparente Herkunft** der Trainingsdaten (keine Black Box).
 - Training auf **europäischer Infrastruktur** – DSGVO-konform, keine Abhängigkeit von US-Cloud.
 - **Open Weights** – Unternehmen können das Modell lokal deployen und anpassen.
 - Starker **Deutschsprachfokus** im Pretraining – nicht nur als Afterthought übersetzt.
- **Hybride Architektur:** Mamba-2/MoE statt reinem Transformer – aktive Forschung statt Kopie.
- **Zielgruppen:** Industrie (Automotive, Recht, Verwaltung), Behörden, Forschung – überall wo Datensouveränität kritisch ist.
- **Roadmap:** Weitere Modelle (Dialog, Reasoning, Agents) geplant; industrielle Pilotprojekte 2026.

<https://soofi.info> | <https://huggingface.co/Soofi-Project> | arXiv:2607.09424



Einführung in Reasoning-Modelle

- Reasoning-Modelle zielen darauf ab, logisches Denken und Schlussfolgerungen zu ermöglichen.
- Fokus auf komplexe Aufgaben wie mathematische Beweise, logische Schlussfolgerungen und Multi-Hop-Fragen.
- Beispiele: DeepSeek-r1, GPT4o, DeepMind Gemini, Anthropic Claude.

DeepSeek-r1: Überblick

■ **DeepSeek-r1:** Ein neuartiges Reasoning-Modell, das auf Transformer-Architekturen basiert.

■ **Ziele:**

- Integration von logischem Denken in Sprachmodelle.
- Dieses Reasoning imitiert menschliches Denken über Generierung von Tokens
- Verarbeitung von Multi-Hop-Reasoning-Aufgaben.
- Unterstützung von domänenspezifischen Schlussfolgerungen.

■ **Besonderheiten:**

- Hybrid-Architektur mit dedizierten Reasoning-Modulen.
- Nutzung von Memory-Augmented Mechanismen.

DeepSeek-r1: Architektur

■ Core-Komponenten:

- **Reasoning-Module:** Spezialisierte Submodule für logische Schlussfolgerungen.
- **Memory-Augmented Attention:** Ermöglicht Zugriff auf externe Wissensquellen.
- **Multi-Hop-Mechanismus:** Iterative Verarbeitung von Informationen.

■ Pipeline:

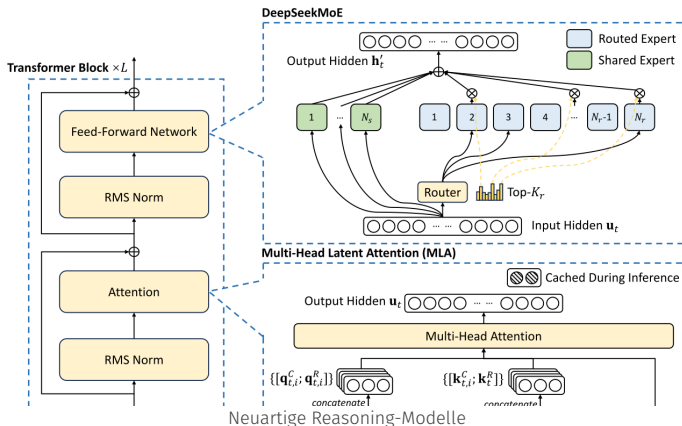
1. Eingabe wird tokenisiert und in Embeddings umgewandelt.
2. Reasoning-Module führen "logische" Operationen durch.
3. Ergebnisse werden iterativ verfeinert.
4. Ausgabe erfolgt als Schlussfolgerung oder Antwort.

Überblick DeepSeek-R1

- **Ziel:** Maximale Ausnutzung von Test-Time Computation zur Förderung von Ketten- bzw. Chain-of-Thought (CoT) Reasoning.
- DeepSeek-R1 erreicht ähnliche Leistungen wie GPT-o1, jedoch mit deutlich geringeren Trainingskosten.
- Einsatz von Reinforcement Learning (RL) als dominanter Post-Training-Ansatz, um komplexe Reasoning-Aufgaben (Mathematik, Code, wissenschaftliches Denken) zu meistern.

Architektur von DeepSeek-R1

- Basierend auf einem DeepSeek-V3-Base Checkpoint.
- Nutzt **Mixture-of-Expert (MoE)**-Strukturen, Multi-Head Latent Attention (MLA) und Multi-Token Prediction (MTP).
- Zwei Varianten:
 - **DeepSeek-R1-Zero**: Post-Training ausschließlich mit RL.
 - **DeepSeek-R1**: Mehrstufiger Post-Training-Prozess, der zusätzlich Supervised Fine-Tuning (SFT) integriert.



Proximal Policy Optimization (PPO) - Teil 1

- **Definition:** PPO ist ein Reinforcement-Learning-Algorithmus, der die Policy-Gradient-Methode verbessert.
- **Ziel:** Maximierung der kumulierten Belohnung durch Optimierung der Policy π_{θ} .
- **Kernidee:** Begrenzung der Policy-Änderungen, um Stabilität und Effizienz zu gewährleisten.
- **Details:** <https://arxiv.org/pdf/2402.03300>

PPO-Zielfunktion:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

Proximal Policy Optimization (PPO) - Teil 2

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$: Verhältnis der neuen zur alten Policy.
- $\hat{A}_t$: Vorteil (Advantage) zur Zeit t .
- ϵ : Hyperparameter zur Begrenzung der Policy-Änderung.

Vorteile von PPO:

- Stabilität durch Clipping der Policy-Änderungen.
- Einfache Implementierung und gute Leistung in verschiedenen RL-Aufgaben.

GRPO – Group Relative Policy Optimization (Teil 1)

- DeepSeek-R1 setzt auf eine modifizierte Version von PPO namens **Group Relative Policy Optimization (GRPO)**.
- Ziel: Steigerung der mathematischen Reasoning-Fähigkeiten bei reduziertem Speicherverbrauch.

GRPO Objective:

Die GRPO-Ziel-Funktion basiert auf der PPO-Formulierung:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)]$$

wobei:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ ist der Wahrscheinlichkeitsquotient.
- $\hat{A}_t$ bezeichnet den Vorteil (Advantage) zur Zeit t .
- ϵ ist ein Hyperparameter zur Begrenzung der Änderung.

GRPO – Group Relative Policy Optimization (Teil 2)

Gruppenbezogene Anpassungen:

GRPO erweitert die PPO-Zielfunktion, indem es gruppenspezifische Gewichtungen einführt:

$$L^{\text{GRPO}}(\theta) = \mathbb{E}_{t,g} \left[\min \left(r_{t,g}(\theta) \hat{A}_{t,g}, \text{clip} \left(r_{t,g}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{t,g} \right) \right]$$

wobei:

- g die Gruppe repräsentiert, zu der die Aufgabe gehört.
- $r_{t,g}(\theta) = \frac{\pi_{\theta}(a_t|s_t,g)}{\pi_{\theta_{\text{old}}}(a_t|s_t,g)}$ ist der gruppenspezifische Wahrscheinlichkeitsquotient.
- $\hat{A}_{t,g}$ ist der gruppenspezifische Vorteil (Advantage).

Diese Erweiterung ermöglicht es, die Policy-Optimierung an die spezifischen Anforderungen und Eigenschaften verschiedener Gruppen anzupassen, wodurch die Leistung in heterogenen Aufgabenbereichen verbessert wird.

GRPO – Group Relative Policy Optimization (Teil 3)

- **Visualisierung:** Die folgende Abbildung illustriert die Funktionsweise von GRPO.
- **Schlüsselkonzepte:**
 - Gruppenspezifische Gewichtungen.
 - Begrenzung der Policy-Änderungen.
 - Iterative Optimierung für verschiedene Gruppen.

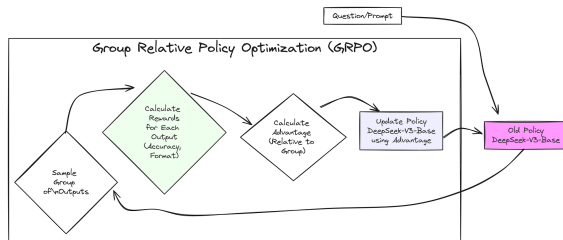


Abbildung: Illustration der GRPO-Mechanik.

Reward-Modellierung

■ Regelbasierte Rewards:

- **Accuracy Reward:** Bewertet, ob die Antwort korrekt ist.
- **Format Reward:** Erzwingt, dass der Denkprozess in `<think>` und `</think>` Tags eingeschlossen wird.
- Kein neutral trainierter Reward Model, da solche Modelle anfällig für Reward Hacking sind.

Kombinierter Reward:

$$R_{\text{total}} = \alpha R_{\text{accuracy}} + \beta R_{\text{format}}$$

- α und β sind Gewichtungsfaktoren, die den Einfluss der jeweiligen Komponenten steuern.

Mehrstufiger Trainingsprozess von DeepSeek-R1

1. Phase 1: Cold-Start SFT

- Erste Supervised Fine-Tuning (SFT) Phase, bei der Labels durch wenige Beispiele von R1-Zero generiert und von Menschen verfeinert werden.

2. Phase 2: Reinforcement Learning

- Anwendung von GRPO zur Optimierung der Reasoning-Fähigkeiten.
- Mathematische Zielsetzung zur Maximierung der Test-Time Computation: Der durchschnittliche Antwortlänge-Wert steigt, was eine tiefergehende Ketten-Denke (Chain-of-Thought) anzeigt.

3. Phase 3: Weitere SFT

- Integration von Daten aus weiteren Domänen zur Verbesserung von Schreibstil, Rollenspiel und allgemeinen Aufgaben.

4. Phase 4: Sekundäres RL

- RL für alle Szenarien zur Steigerung der Hilfsbereitschaft und Harmlosigkeit.

Chain-of-Thought und Test-Time Computation

- DeepSeek-R1 nutzt **Chain-of-Thought (CoT)**-Reasoning:
 - Zuerst wird ein ausführlicher Denkprozess (innerhalb von `<think> ... </think>` Tags) generiert.
 - Anschließend wird die finale Antwort produziert.
- Mathematische Modellierung dieser Phase kann als iterative Optimierung über Teilschritte betrachtet werden, z.B.:

$$\mathbf{c}_{t+1} = f(\mathbf{c}_t, \Delta_t(\mathbf{x}))$$

wobei $\mathbf{c}_t$ den aktuellen CoT-Zustand und $\Delta_t(\mathbf{x})$ den Beitrag des nächsten Tokens bzw. der nächsten Denkschritt repräsentiert.

- Durch längeres Denken bei steigender Testzeit skaliert die Modellleistung im Sinne der "Test-time Scaling Law".

Zusammenfassung

- DeepSeek-R1 demonstriert, wie Reinforcement Learning (GRPO) und regelbasierte Reward-Strategien genutzt werden können, um komplexe reasoning-Aufgaben zu bewältigen.
- Der mehrstufige Trainingsprozess (SFT $\rightarrow$ RL $\rightarrow$ SFT $\rightarrow$ RL) verbessert sowohl die Genauigkeit als auch die Sprachkohärenz.
- Die mathematische Grundlage der GRPO-Ziel-Funktion und der Reward-Modellierung zeigen, wie Optimierungsziele systematisch in den Trainingsprozess integriert werden.

DeepSeek-R1 zeigt: Mit reduzierten Trainingskosten und innovativen Trainingsansätzen sind moderne Reasoning-Modelle möglich.

Zusammenfassung: DeepSeek-r1 und LLM-Architekturen

■ DeepSeek-r1:

- Hervorragende Reasoning-Fähigkeiten und Multi-Hop-Reasoning.
- Zugriff auf externen Speicher für komplexe Schlussfolgerungen.
- Anwendungen: Logik, Beweise, domänenspezifische Aufgaben.

■ Vergleich von LLMs:

- GPT-2: Verbesserte Textgenerierung, Anwendungen wie Textzusammenfassung.
- GPT-3: 175B Parameter, Few-Shot Learning, z. B. Codegenerierung.
- GPT-4: Multimodalität (Text und Bild), Anwendungen wie Bildbeschreibung.
- LLaMA: Open-Weight Modelle (7B-70B Parameter), Fokus auf Forschung.
- DeepSeek-v3: Fortschrittliche Reasoning- und Domänenanpassungsfähigkeiten, Anwendungen in Wissenschaft und Technik.



Und was machen wir jetzt mit diesen Systemen?

- Praktische Anwendungen und Integration in bestehende Systeme
- Herausforderungen bei der Implementierung und Skalierung
- Gesellschaftliche und ethische Implikationen

Nutzungsmöglichkeiten: RAG, Agentensysteme

- Retrieval-Augmented Generation (RAG) und seine Anwendungen
- Entwicklung und Einsatz von Agentensystemen im NLP
- Kombination von LLMs mit externem Wissen

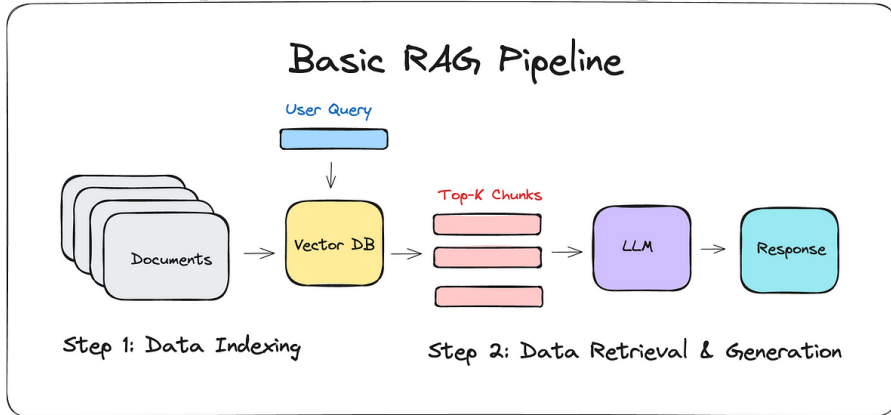
Was ist Retrieval-Augmented Generation (RAG)?

- **Definition:** Kombination von Retrieval-Systemen und generativen Modellen.
- **Ziel:** Verbesserung der Antwortqualität durch Zugriff auf externe Wissensquellen.
- **Funktionsweise:**
 - Abruf relevanter Dokumente aus einer Wissensdatenbank.
 - Nutzung der abgerufenen Informationen zur Generierung von Antworten.
- **Anwendungen:** Fragebeantwortung, Chatbots, Dokumentensuche.

RAG: Architektur

■ Retriever:

- Abruf relevanter Dokumente aus einer Wissensdatenbank.
- Nutzung von Suchalgorithmen und Vektorraumsmodellen.



RAG: Vorteile

- **Verbesserte Genauigkeit:** Zugriff auf externe Wissensquellen reduziert Halluzinationen.
- **Flexibilität:** Kann mit verschiedenen Retrieval- und Generationsmodellen kombiniert werden.
- **Aktualität:** Ermöglicht die Nutzung aktueller Informationen aus einer Wissensdatenbank.
- **Erweiterbarkeit:** Einfaches Hinzufügen neuer Wissensquellen.

RAG: Herausforderungen

- **Effizienz:** Abruf und Generierung können rechenintensiv sein.
- **Schnittstellen:** Legacy-Systeme und APIs müssen integriert werden.
- **Access Rights Management:** Sicherstellen, dass nur autorisierte Informationen abgerufen werden.
- **Qualität der Dokumente:** Die Genauigkeit hängt von der Qualität der abgerufenen Dokumente ab.
- **Konsistenz:** Inkonsistenzen zwischen abgerufenen Informationen und generierten Antworten.
- **Bias:** Bias in der Wissensdatenbank können die Antworten beeinflussen.

RAG: Anwendungen

- **Fragebeantwortung:** Beantwortung komplexer Fragen durch Abruf relevanter Informationen.
- **Chatbots:** Verbesserung der Konversationsqualität durch Zugriff auf externe Daten.
- **Dokumentensuche:** Generierung von Zusammenfassungen basierend auf abgerufenen Dokumenten.
- **Wissenschaftliche Recherche:** Unterstützung bei der Suche nach relevanter Literatur.

Was ist Document Embedding in RAG Pipelines?

- **Definition:** Repräsentation eines gesamten Dokuments/Chunks als Vektor in einem kontinuierlichen Vektorraum, speziell für Retrieval-Augmented Generation (RAG).
- **Ziel:** Ermöglichen eines effizienten Abrufs relevanter Dokumente/Chunks aus einer Vektordatenbank.
- **Anwendungen in RAG:**
 - Verbesserung der Antwortqualität durch Zugriff auf relevante Dokumente.
 - Unterstützung von Fragebeantwortung und Dokumentensuche.
 - Integration von externem Wissen in generative Modelle.

Strategien für Document Embedding in RAG Pipelines

■ Transformer-basierte Modelle:

- Nutzung von Modellen wie Sentence-BERT, OpenAI Embeddings oder ähnliche.
- CLS-Token oder Mittelung der Token-Embeddings für die Dokumentrepräsentation.
- Vorteile: Kontextabhängige und semantisch reichhaltige Repräsentationen.

■ Vektordatenbanken:

- Speicherung der Dokumentvektoren in spezialisierten Datenbanken wie Pinecone, Weaviate oder Milvus.
- Ermöglichen schnellen Abruf durch Ähnlichkeitssuche (z. B. k-NN, cosine similarity).

■ Hybrid-Ansätze:

- Kombination von klassischen Retrieval-Methoden (z. B. BM25) mit Vektorbasierter Suche.
- Verbesserung der Präzision durch Kombination von Semantik und Schlüsselwortsuche.

Einbettung in Vektordatenbanken für RAG Pipelines

■ Pipeline:

1. Dokumente werden vorverarbeitet und in Vektoren eingebettet.
2. Die Vektoren werden in einer Vektordatenbank gespeichert.
3. Bei einer Anfrage wird der Eingabetext ebenfalls eingebettet.
4. Ähnlichkeitssuche in der Vektordatenbank liefert relevante Dokumente.
5. Die abgerufenen Dokumente werden als Kontext für die Generierung verwendet.

■ Vorteile:

- Effiziente Suche in großen Wissensbasen.
- Kontextualisierte Antworten durch semantische Relevanz.
- Skalierbarkeit für umfangreiche Datenmengen.

Hybride Suche: BM25 und Vektorähnlichkeiten (Teil 1)

■ **Definition:** Kombination von klassischen Suchmethoden (BM25) und vektorbasierter Ähnlichkeitssuche.

■ **BM25:**

- Klassischer Algorithmus für die Schlüsselwortsuche.
- Bewertet die Relevanz eines Dokuments basierend auf Termfrequenz (TF) und inverser Dokumentfrequenz (IDF).
- Formel:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})}$$

wobei $f(t, d)$ die Häufigkeit des Terms t im Dokument d ist.

■ **Vektorähnlichkeiten:**

- Repräsentiert Dokumente und Anfragen als Vektoren in einem kontinuierlichen Raum.
- Nutzt Metriken wie Kosinus-Ähnlichkeit oder euklidische Distanz zur Bewertung der Relevanz.

Hybride Suche: BM25 und Vektorähnlichkeiten (Teil 2)

■ Kombination:

- BM25 liefert eine gewichtete Bewertung basierend auf Schlüsselwörtern.
- Vektorähnlichkeiten ergänzen die Suche durch semantische Relevanz.
- Hybride Bewertung:

$$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{BM25}(q, d) + \beta \cdot \text{Similarity}(q, d)$$

wobei α und β Gewichtungsfaktoren sind.

■ Vorteile:

- Präzision durch BM25 für Schlüsselwortsuche.
- Semantische Tiefe durch Vektorähnlichkeiten.
- Flexibilität für verschiedene Anwendungsfälle.

■ Anwendungen:

- Dokumentensuche in großen Datenbanken.
- Fragebeantwortungssysteme mit externem Wissen.
- Kombination von strukturierten und unstrukturierten Daten.

Vergleich der Strategien für RAG Pipelines

Methode	Vorteile	Nachteile
BM25	Schnell, etabliert	Keine Semantik
Transformer-Modelle	Kontext- und Semantikreich	Hoher Rechenaufwand
Vektordatenbanken	Effiziente Ähnlichkeitssuche	Speicherbedarf
Hybrid-Ansätze	Kombination von Präzision und Semantik	Komplexität

Tabelle: Vergleich verschiedener Strategien für RAG Pipelines.

Grenzen von dokumentenbasiertem Retrieval – Motivation

- Klassisches RAG ruft Textpassagen ab – Beziehungen zwischen Entitäten bleiben **implizit im Fließtext**.
- **Konkrete Probleme:**
 - **Terminologische Inkonsistenz:** “Herzinfarkt”, “Myokardinfarkt” und “AMI” meinen dasselbe – der Retriever weiß das nicht.
 - **Multi-hop-Fragen:** “Welche Nebenwirkungen hat Medikament X bei Patienten mit Diagnose Y?” erfordert das Traversieren von Beziehungen, nicht nur Textsuche.
 - **Keine Domänenlogik:** Textchunks kennen keine Taxonomien, Regeln oder Axiome.
- **Lösungsweg:** Text schrittweise in strukturiertes Wissen überführen.
 1. Entitäten erkennen → **Named Entity Recognition (NER)**
 2. Beziehungen extrahieren → **Relationsextraktion**
 3. Auf formales Vokabular abbilden → **Ontologie-Mapping**
 4. Im Graphen speichern und abfragen → **Wissensgraph + GraphRAG**

Was ist Named Entity Recognition (NER)?

- **Definition:** NER identifiziert und klassifiziert benannte Entitäten im Text – der **erste Schritt** von unstrukturiertem Text zu strukturiertem Wissen.
- **Typische Entitätstypen (CoNLL-2003 Standard):**
 - **PER** – Personen (z. B. “Angela Merkel”)
 - **ORG** – Organisationen (z. B. “SAP AG”)
 - **LOC** – Orte und Lokationen (z. B. “Berlin”)
 - **DATE/TIME** – Zeitangaben (z. B. “März 2024”)
 - **MISC** – Sonstiges (Produkte, Ereignisse, Sprachen)
- **Domänenspezifische Erweiterungen:**
 - Medizin: GENE, DISEASE, DRUG, PROCEDURE (BC5CDR, NCBI Disease)
 - Recht: LAW, COURT, JUDGE, REGULATION
 - Industrie: MACHINE, SENSOR, COMPONENT, PROCESS

NER-Methoden: Von Regelbasiert zu Transformer

■ Regelbasierte Ansätze:

- Wörterbücher, reguläre Ausdrücke und handgefertigte Grammatiken.
- Präzise, aber unflexibel und aufwendig in der Pflege.

■ ML-basierte Ansätze (CRF):

- Conditional Random Fields (CRF) modellieren Abhängigkeiten zwischen Tags in einer Sequenz.
- IOB-Tagging: B-PER, I-PER, O (Begin, Inside, Outside).

■ Transformer-basierte Ansätze:

- BERT als Token-Klassifikator: Jeder Token erhält ein NER-Label.
- State-of-the-Art durch kontextualisierte Embeddings.
- Mehrsprachig: mBERT, XLM-RoBERTa.

BERT für NER: Token-Klassifikation

- **Ansatz:** BERT wird auf Token-Ebene feinjustiert – jeder Token erhält eine Entitätsklasse.

- **IOB-Schema Beispiel:**

Angela [B-PER] Merkel [I-PER] war [O] Bundeskanzlerin [O] Deutschlands [B-LOC].

- **Fine-Tuning:**

- Linearer Klassifikationslayer auf BERT-Token-Embeddings.
- Training auf beschrifteten Datensätzen (z. B. CoNLL-2003, GermEval).

- **Tools:** spaCy (Transformer-Backend), Hugging Face `token-classification`, Flair.

Relationsextraktion und Koreferenzauflösung

■ Relationsextraktion (RE):

- Erkennt semantische Beziehungen zwischen erkannten Entitäten.
- Beispiel: "Bosch hat seinen Sitz in Stuttgart."
⇒ (Bosch, headquartered_in, Stuttgart)
- Methoden: Pipeline (NER → RE), End-to-End-Transformer-Modelle.

■ Koreferenzauflösung:

- Identifikation von Ausdrücken, die auf dieselbe Entität verweisen.
- Beispiel: "Elon Musk gründete Tesla. **Er** ist auch CEO von SpaceX."
- Wichtig für korrekte Entitätsverfolgung über längere Texte.

- **Relevanz:** NER + RE + Koreferenz bilden die Grundlage für automatischen Wissensgraphaufbau.

Hybride Pipeline: Text → Entitäten → Wissensgraph (Teil 1)

- **Ziel:** Unstrukturierter Text wird schrittweise in einen maschinenlesbaren, abfragbaren Wissensgraphen überführt.
- **Pipeline-Schritte:**
 1. **Textvorbereitung:** Satzgrenzerkennung, Tokenisierung, Normalisierung.
 2. **NER:** Erkennung von Entitätserwähnungen (PER, ORG, LOC, DISEASE, ...).
 3. **Koreferenzauflösung:** Zusammenführen von Erwähnungen derselben Entität.
 4. **Relationsextraktion:** Erkennung von Tripeln (Subjekt, Prädikat, Objekt).
Bsp.: (Metformin, behandelt, DiabetesTyp2)
 5. **Entity Linking:** Mapping auf eindeutige Ontologie-Konzept-IDs.
Bsp.: "Herzinfarkt" ↦ SNOMED:22298006
 6. **Graphaufbau:** Knoten = verlinkte Konzepte, Kanten = typisierte Relationen.

Hybride Pipeline: Typische Herausforderungen (Teil 2)

- **Entity Disambiguation:** “Apple” – Unternehmen oder Frucht?
⇒ Kontextbasiertes Linking durch Bi-Encoder/Cross-Encoder.
- **Ontologie-Lücken (erwartet und normal!):** Wachsende Domänen bringen ständig neue Konzepte – Ontologien werden versioniert wie Software.
⇒ NIL-Detektion flaggt unbekannte Entitäten; Ontologieexperten ergänzen sie in der nächsten Ontologieversion. Mit der aktualisierten Ontologie wird der Graph inkrementell neu aufgebaut.
- **Fehlerfortpflanzung:** NER-Fehler → falsche Relationen → falsches Graphwissen.
⇒ Konfidenz-Scores, Stichprobenprüfung, Revisions-Pipelines.
- **Skalierbarkeit:** Millionen Dokumente erfordern Streaming-Verarbeitung (z. B. Spark NLP).
- **Mehrsprachigkeit:** Entitäten verschiedener Sprachen werden auf dieselbe Ontologie abgebildet.
⇒ Mehrsprachige Verlinkungsmodelle (mGENRE, XLM-RoBERTa-basiert).

Werkzeuge: spaCy • Hugging Face • BLINK • Neo4j • Microsoft GraphRAG²¹ • LlamaIndex

²¹<https://github.com/microsoft/graphrag>

Was ist eine Ontologie? – Grundlagen

- **Definition (Informatik):** Eine Ontologie ist eine formale, maschinenlesbare Spezifikation einer geteilten Konzeptualisierung – sie definiert präzise, was in einer Domäne existiert und wie Konzepte zusammenhängen.
- **Vier Kernkomponenten:**
 - **Klassen (Concepts):** Abstrakte Kategorien, z. B. `schema:Organization`, `cmns:Party`.
 - **Properties (Relationen):** Typisierte Verbindungen, z. B. `schema:founder`, `cmns:isPartyTo`.
 - **Instanzen (Individuals):** Konkrete Objekte, z. B. `ex:W3C` $\in$ `schema:Organization`.
 - **Axiome (Restrictions):** Logische Regeln, z. B.
`cmns:LegalEntity` $\sqsubseteq$ `cmns:Party`
- **W3C-Standards:** OWL 2 (Web Ontology Language), RDF/RDFS, SKOS – offene Standards des World Wide Web Consortium.²²
- **Abfragesprache:** SPARQL – SQL für Graphen und Ontologien.
Bsp.: `SELECT ?grnd WHERE { ex:W3C schema:founder ?grnd }`

²²<https://www.w3.org/standards/semanticweb/>

Ontologien: Competency Questions – Anforderungen formalisieren

- **Competency Questions (CQs)** sind natürlichsprachliche Fragen, die eine Ontologie beantworten können soll – sie definieren Umfang und Anforderungen eines Ontologieprojekts.²³
- **Zweck:**
 - Steuern, welche Konzepte und Relationen modelliert werden müssen (Scope).
 - Dienen zur **Evaluation**: Kann die Ontologie alle CQs via SPARQL beantworten?
 - Verhindern Over-Engineering: Konzepte ohne CQ-Deckung gehören nicht in die Ontologie.
- **Beispiel-CQs (Unternehmensontologie, z. B. OMG FIBO / Commons²⁴):**
 - “Welche juristischen Personen sind direkte Tochtergesellschaften von Unternehmen X?”
 - “Welche Vertragsparteien sind an einem Kreditvertrag beteiligt?”
 - “In welchen Ländern ist eine Organisation rechtsverbindlich registriert?”
- **Methoden:** METHONTOLOGY, NeOn Methodology – systematische Ontologieentwicklung mit CQs als rotem Faden.

²³Gruber, T. R. (1993). A translation approach to portable ontology specifications. Knowledge Acquisition, 5(2), 199–220.

²⁴<https://www.omg.org/spec/Commons>

Ontologien: Menschliche Kuration und etablierte Beispiele

- **Wichtig:** Ontologien werden primär von **Domänenexperten und Fachgesellschaften manuell erstellt und kuratiert**. Sie kodieren jahrelang gesammeltes Expertenwissen in maschinenlesbarer, logisch konsistenter Form.
- **Etablierte Ontologien und Standards:**
 - **W3C:** OWL 2, RDF/RDFS, SKOS, SPARQL, FOAF, Dublin Core – offene Webstandards
 - **OMG:** FIBO (>2 000 Finanzkonzepte), Commons Ontology Library (cmns: Parteien, Verträge, Einheiten), LCC (Länder-/Sprachcodes)
 - **Allgemein:** Schema.org (>800 Typen, W3C/Google/Microsoft), Wikidata (>100 Mio. Aussagen), DBpedia
 - **Industrie:** Asset Administration Shell (AAS/RAMI 4.0), GoodRelations (E-Commerce), ISO 15926
 - **Fachdomänen:** Gene Ontology (GO), SNOMED CT, ChEBI – durch Fachgesellschaften kuratiert
- **Pflege und Governance:**
 - Versionierung, Change-Management, Review-Prozesse durch Expertenkomitees.
 - LLMs als Unterstützung bei Erstellung und Pflege – kein autonomer Ersatz.
 - Herausforderung: Schnelllebige Domänen erfordern regelmäßige Aktualisierung.

Vom NER-Ergebnis zur Ontologie: Entity Linking

- **Entity Linking (EL):** Verknüpft einen NER-Textspan mit einem eindeutigen Konzept-URI in einer Ontologie – die Brücke zwischen Text und strukturiertem Wissen.
- **Drei-Schritt-Prozess:**
 1. **Kandidatengenerierung:** Suche ähnlicher Konzeptnamen (TF-IDF, Embedding-Suche).
 2. **Disambiguierung:** Auswahl des korrekten Kandidaten anhand des Satzkontexts (Bi-Encoder für Recall, Cross-Encoder für Precision).
 3. **NIL-Detektion:** Erkennung, wenn kein passender Kandidat existiert.
- **Beispiel:**

Text: “**Bosch** hat seinen Hauptsitz in Stuttgart.”

NER: Bosch → ORGANIZATION

EL + Kontext: Bosch → wd:Q234021 (Robert Bosch GmbH, Wikidata) ≡ cmns:LegalEntity
- **Werkzeuge:** BLINK (Meta), mGENRE (mehrsprachig), REL, ReFinED, GENRE.

Was ist GraphRAG? – Vom Textchunk zum Wissensgraph

- **Definition:** GraphRAG erweitert klassisches RAG: statt Textchunks werden **Subgraphen eines Wissensgraphen** abgerufen – direkt auf Basis der zuvor eingeführten Ontologien.
- **Warum der Schritt?** NER + Relationsextraktion + Entity Linking haben den Text strukturiert – jetzt kann man traversieren statt nur suchen.
- **Vorteile gegenüber klassischem RAG:**
 - Explizite Beziehungen: kein impliziter Fließtext mehr.
 - Multi-hop-Reasoning über Graphpfade möglich.
 - Inferenz: neue Fakten aus vorhandenen Axiomen ableiten.
 - Erklärbarkeit: Antwortpfad im Graph nachvollziehbar.
- **Anwendungen:** Medizin, Recht, Industrie 4.0, wissenschaftliche Recherche, Compliance.²⁵

²⁵Edge et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. <https://arxiv.org/abs/2404.16130>

Ontologien als Rückgrat von GraphRAG

■ **Erinnerung:** Ontologien – primär menschlich kuratiert – bieten ein formales, domänenspezifisches Vokabular (OWL/RDF).

■ **Drei Rollen in GraphRAG:**

■ **Vokabular:** Einheitliche Terminologie verhindert Duplikate und Mehrdeutigkeiten.

■ **Inferenz:** Axiome leiten implizite Fakten automatisch ab.

Bsp.: `Metformin behandelt DiabetesTyp2 + DiabetesTyp2 ist-ein Stoffwechselerkrankung`

⇒ automatisch: `Metformin behandelt (indirekt) Stoffwechselerkrankung`

■ **Interoperabilität:** Verschiedene Datenquellen werden auf eine gemeinsame Ontologie normiert und damit kombinierbar.

■ **Abfrage mit SPARQL:**

```
SELECT ?therapie WHERE { :DiabetesTyp2 :behandeltMit ?therapie }
```


Wissenseinbettung in Graphdatenbanken und Ontologien

- **Graphdatenbank:** Speichert Wissen als Knoten (Entitäten/Konzepte) und Kanten (Beziehungen), oft auf Basis einer Ontologie.
- **Einbettung (Embedding):**
 - Jeder Knoten und jede Kante erhält einen Vektor im kontinuierlichen Raum.
 - Methoden: Node2Vec, GraphSAGE, TransE, GNN-basierte Ansätze.
- **Pipeline:**
 1. Extraktion von Entitäten, Relationen und Konzepten aus Text (z. B. mit NLP und Ontologie-Mapping).
 2. Aufbau des Wissensgraphen in einer Graphdatenbank (z. B. Neo4j, TigerGraph) unter Nutzung einer Ontologie.
 3. Berechnung von Embeddings für Knoten/Kanten.
 4. Retrieval relevanter Subgraphen/Konzepte als Kontext für LLMs.

Beispiel 1: Medizinische Fragebeantwortung mit Ontologie

- **Ontologie:** SNOMED CT (medizinische Begriffe und Relationen)
- **Frage:** "Welche Therapien gibt es für Diabetes Typ 2?"
- **Ablauf:**
 1. LLM erkennt Entität "Diabetes Typ 2" und mapped sie auf die Ontologie.
 2. GraphRAG sucht im Wissensgraphen nach Knoten "Diabetes Typ 2" und allen Kanten "behandelt mit".
 3. Die gefundenen Therapien werden als strukturierter Kontext an das LLM übergeben.
 4. LLM generiert eine nachvollziehbare, medizinisch fundierte Antwort.
- **Vorteil:** Medizinische Präzision, Nachvollziehbarkeit, Nutzung von Expertenwissen.

Beispiel 2: Industrie 4.0 – Asset-Verwaltung mit Ontologie

- **Ontologie:** Asset Administration Shell (AAS) Ontologie
- **Frage:** "Welche Maschinen sind Teil der Fertigungslinie X und wann ist die nächste Wartung?"
- **Ablauf:**
 1. LLM mapped "Fertigungslinie X" auf die Ontologie.
 2. GraphRAG sucht alle Maschinen (Knoten) mit Relation "ist Teil von" zu "Fertigungslinie X".
 3. Für jede Maschine wird die Relation "nächste Wartung" abgefragt.
 4. LLM gibt eine strukturierte Übersicht aus.
- **Vorteil:** Transparenz, Automatisierung, Integration von Echtzeitdaten.

GraphRAG: Architekturüberblick

■ Komponenten:

- **Graphdatenbank/Ontologie:** Speicherung und Abfrage von Wissensgraphen und Konzepten.
- **Graph Embedding-Modell:** Erzeugt Vektorrepräsentationen für Knoten/Kanten/Subgraphen.
- **Retriever:** Findet relevante Subgraphen/Konzepte zu einer Anfrage.
- **LLM:** Nutzt die abgerufenen Graph- und Ontologiekontexte zur Antwortgenerierung.

■ Ablauf:

1. Anfrage wird in Entitäten/Relationen/Konzepte zerlegt (NLP + Ontologie-Mapping).
2. Passende Subgraphen/Konzepte werden per Embedding-Ähnlichkeit oder Graphabfrage gefunden.
3. Kontext wird an das LLM übergeben.
4. LLM generiert Antwort unter Nutzung des strukturierten Wissens.

Vergleich: Klassisches RAG vs. GraphRAG mit Ontologien

Eigenschaft	Klassisches RAG	GraphRAG mit Ontologie
Wissensquelle	Dokumente/Chunks	Wissensgraph/Ontologie
Kontextabruf	Vektorähnlichkeit	Graphabfragen + Embeddings + Inferenz
Beziehungen	Implizit im Text	Explizit und semantisch modelliert
Reasoning	Eingeschränkt	Komplexe Pfad-, Beziehungs- und Konzeptabfragen
Erklärbarkeit	Gering	Sehr hoch
Domänenwissen	Unstrukturiert	Standardisiert, interoperabel

Tabelle: Vergleich von klassischem RAG und GraphRAG mit Ontologien.

Aktuelle GraphRAG-Varianten und Forschungsstand

- **Microsoft GraphRAG**²⁶ (2024): Hierarchische Community-Strukturen im Wissensgraphen; getrennte globale und lokale Abfragestrategien.
- **HybridRAG**²⁷ (2024, BlackRock/NVIDIA): Kombination von Vektor-Retrieval und KG-Abfragen – erzielt bessere Präzision als beide Einzelansätze.
- **LightRAG**²⁸ (2025, EMNLP): Vereinfachtes GraphRAG mit schnellerer Graph-Konstruktion, dual-level Retrieval.
- **KG-guided RAG**²⁹ (2025, NAACL): Ontologiegesteuerte Abfragepläne verbessern Multi-hop-Reasoning.
- **Übersicht:** Peng et al. (2025). Graph Retrieval-Augmented Generation: A Survey³⁰ – systematische Kategorisierung von über 100 GraphRAG-Ansätzen.

²⁶Edge et al. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. <https://arxiv.org/abs/2404.16130>

²⁷Sarmah et al. (2024). HybridRAG: Integrating Knowledge Graphs and Vector Retrieval Augmented Generation.

<https://doi.org/10.1145/3677052.3698671>

²⁸Guo et al. (2025). LightRAG: Simple and Fast Retrieval-Augmented Generation. <https://doi.org/10.18653/v1/2025.findings-emnlp.568>

²⁹Zhu et al. (2025). Knowledge Graph-Guided Retrieval Augmented Generation. <https://doi.org/10.18653/v1/2025.naacl-long.449>

³⁰<https://doi.org/10.1145/3777378>

Herausforderungen und Ausblick für GraphRAG mit Ontologien

■ Herausforderungen:

- Extraktion und Aktualisierung von Entitäten/Relationen/Konzepten aus Text.
- Skalierbare Berechnung und Speicherung von Graph- und Ontologie-Embeddings.
- Effiziente Subgraph- und Konzept-Retrieval-Algorithmen.
- Integration von Ontologiekontext in LLM-Prompts.
- Pflege und Erweiterung von Ontologien.

■ Ausblick:

- Kombination von GraphRAG mit multimodalen Wissensquellen (Text, Bild, Sensorik).
- Automatisierte Ontologie-Generierung und -Aktualisierung.
- Verbesserte Reasoning-Fähigkeiten durch strukturierte, domänenspezifische Kontextbereitstellung.
- Einsatz in kritischen Bereichen wie Medizin, Recht, Industrie.

Von Retrieval zu autonomem Handeln

RAG / GraphRAG (bisher):

- Passives Informationsabrufsystem
- Beantwortet Fragen durch Suchen und Generieren
- Handelt nicht selbst; ändert keine Zustände in der Welt
- Wird als **Tool** von einem Agenten aufgerufen

LLM-Agenten (jetzt):

- Aktiv planend und handelnd
- Nutzen RAG und GraphRAG als **eines von vielen Tools**
- Können APIs aufrufen, Code ausführen, Dateien schreiben
- Iterieren in Zyklen; beobachten und reagieren auf ihre Umgebung
- Koordinieren sich in Multi-Agenten-Systemen

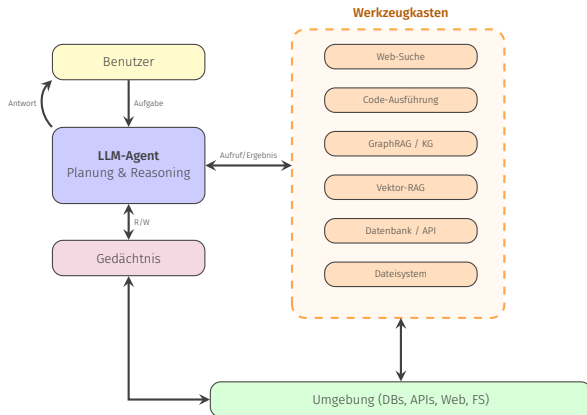
Der Schlüsselunterschied: Agenten **verändern** die Welt – Retrieval-Systeme beschreiben sie nur.

Was sind komplexe Agentensysteme?

- **Definition:** Systeme, die autonome Agenten nutzen, um komplexe Aufgaben durch Interaktion mit Tools, Ontologien und Umgebungen zu lösen.
- **Merkmale:**
 - Autonomie: Agenten agieren teilweise unabhängig in einem vorgegebenen Rahmen.
 - Tool-Nutzung: Zugriff auf externe APIs, Datenbanken, Ontologien oder Software.
 - Multi-Agent-Koordination: Zusammenarbeit mehrerer Agenten.
- **Anwendungen:** Wissenschaftliche Forschung, Automatisierung, Problemlösung, Wissensmanagement.

LLM-Agent: Kernkomponenten und Werkzeugkasten

- **LLM-Kern:** Planung, Reasoning, Entscheidung welches Tool nötig ist
- **Gedächtnis:**
 - Kurzzeit: Kontextfenster
 - Langzeit: Vektorspeicher, Skills
- **Werkzeugkasten:** Dynamisch erweiterbar
 - RAG ist eines von vielen Tools
- **Umgebung:** Agenten beobachten und verändern aktiv die Welt



Beispiel: Multi-Agentensystem für medizinische Diagnose

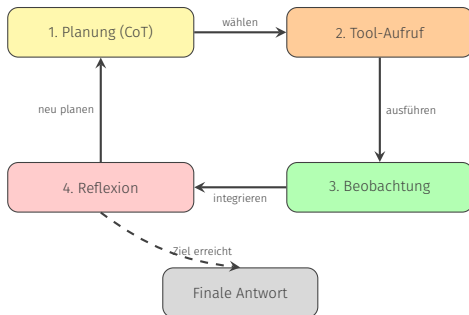
- **Ziel:** Automatisierte Unterstützung bei der medizinischen Diagnose.
- **Agentenrollen:**
 - **Symptom-Analyseagent:** Extrahiert Symptome aus Patientendaten.
 - **Ontologie-Agent:** Nutzt medizinische Ontologien (z.B. SNOMED CT), um Symptome mit möglichen Diagnosen zu verknüpfen.
 - **Literaturagent:** Sucht nach aktuellen Studien zu den gefundenen Diagnosen.
 - **Berichtsagent:** Generiert einen strukturierten Diagnosebericht.
- **Tool-Nutzung:**
 - Zugriff auf medizinische Datenbanken und Ontologien.
 - Nutzung von NLP und LLMs zur Kontextanreicherung.
- **Vorteile:** Präzision, Nachvollziehbarkeit, Integration von Expertenwissen.

Beispiel: Multi-Agentensystem für wissenschaftliche Forschung

- **Ziel:** Automatisierte Literaturrecherche und Datenanalyse.
- **Agentenrollen:**
 - **Suchagent:** Durchsucht Datenbanken nach relevanten Artikeln.
 - **Beispiele für Datenbanken:** PubMed, ArXiv, Semantic Scholar, SpringerLink, IEEE Xplore.
 - **Ontologie-Agent:** Ordnet gefundene Artikel zu Konzepten einer wissenschaftlichen Ontologie (z. B. Gene Ontology).
 - **Analyseagent:** Führt Analysen zu den gefundenen Dokumenten durch.
 - **Berichtsagent:** Generiert Zusammenfassungen und Berichte.
- **Tool-Nutzung:**
 - Zugriff auf APIs, Ontologien und LLMs.
 - Nutzung von Python-Bibliotheken wie Pandas oder Matplotlib.
- **Vorteile:** Effizienzsteigerung, strukturierte Ergebnisse, semantische Suche.

Der Agentic Loop: Planung, Handeln, Beobachten, Reflektieren

- Agenten arbeiten in **iterativen Zyklen** bis das Ziel erreicht ist
- **Planungsstrategien:** Chain-of-Thought, Tree of Thoughts^a, Task Decomposition
- **Abbruch:** bei Zielerreichung, erkanntem Fehler oder erreichtem Iterations-Limit
- **Typische Herausforderungen:**
 - Kontextfensterlimit bei langen Loops
 - Fehlerfortpflanzung in mehrstufigen Pipelines
 - Endlosschleifen (Loop-Erkennung) und unkontrollierte Tool-Aufrufe



^aYao et al. (2023). Tree of Thoughts: Deliberate Problem Solving with LLMs. <https://arxiv.org/abs/2305.10601>

ReAct: Reasoning und Acting – Sichtbares Denken

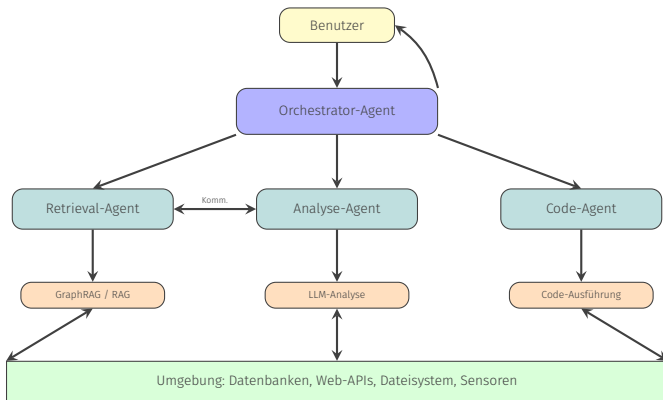
- **ReAct**³¹: Das Modell wechselt explizit zwischen Gedankenschritten und Tool-Aktionen.
- **Vorteil**: Beobachtungen (Tool-Ergebnisse) werden sofort in den nächsten Denkschritt integriert – Fehler sind früh korrigierbar.
- **Verwandte Ansätze**: Self-Ask, Chain-of-Thought + Tools, Reflexion³².



³¹Yao et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. <https://arxiv.org/abs/2210.03629>

³²Shinn et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. <https://arxiv.org/abs/2303.11366>

Multi-Agenten-Architektur: Orchestrierung und Umgebungsinteraktion



Herausforderungen bei komplexen Agentensystemen

- **Koordination:** Synchronisation zwischen mehreren Agenten.
- **Tool- und Ontologie-Integration:** Kompatibilität mit verschiedenen APIs, Ontologien und Software.
- **Fehlerbehandlung:** Umgang mit Ausfällen oder unvorhergesehenen Ereignissen.
- **Skalierbarkeit:** Effizienz bei wachsender Anzahl von Agenten oder Aufgaben.
- **Sicherheit:** Schutz vor Missbrauch oder fehlerhaften Aktionen.
- **Ontologiepflege:** Aktualisierung und Erweiterung der Wissensbasis.

Was ist Tool Calling mit MCP-Servern?

- **Tool Calling:** LLMs rufen externe Tools oder APIs auf, um Aufgaben zu lösen, die über reine Textgenerierung hinausgehen.
- **MCP-Server (Multi-Component Platform):** Vermittlungsinstanz, die Anfragen von LLMs entgegennimmt, an spezialisierte Tools weiterleitet und die Ergebnisse zurückgibt.
- **Ziel:** Erweiterung der Fähigkeiten von LLMs durch strukturierte, sichere und skalierbare Tool-Nutzung.

Model Context Protocol (MCP): Überblick

- **Definition:** Das Model Context Protocol (MCP) ist ein standardisiertes Protokoll zur strukturierten Kommunikation zwischen LLMs, Agenten und externen Tools.
- **Ziel:** Ermöglicht die Übergabe von Kontext, Aufgaben, Tool-Aufrufen und Ergebnissen in einem maschinenlesbaren Format (z. B. JSON, YAML).
- **Kernfunktionen:**
 - **Kontextübergabe:** Übermittlung von Hintergrundwissen, Benutzeranfragen und Umgebungsdaten an das Modell.
 - **Tool-Calls:** Strukturierte Anforderung von externen Aktionen (z. B. Datenbankabfragen, API-Aufrufe).
 - **Antwortintegration:** Rückgabe von Tool-Ergebnissen und deren Einbindung in die Modellantwort.
 - **Status- und Fehlerhandling:** Standardisierte Rückmeldungen zu Erfolg, Fehlern und Zwischenergebnissen.
- **Beispielstruktur (JSON):**
 - `{"context": {}, "task": "...", "tool_calls": [...], "results": [...], "status": "ok"}`
- **Vorteile:** Interoperabilität, Nachvollziehbarkeit, Modularität und sichere Integration von LLMs in komplexe Agentensysteme.

Architektur: Tool Calling mit MCP-Servern

■ Komponenten:

- **LLM:** Erkennt, wann ein Tool-Aufruf nötig ist, und formuliert eine strukturierte Anfrage.
- **MCP-Server:** Vermittelt zwischen LLM und Tools, verwaltet Authentifizierung, Logging und Fehlerbehandlung.
- **Tools/Services:** Externe APIs, Datenbanken, Rechenmodule oder Ontologien.

■ Ablauf:

1. LLM generiert eine Tool-Call-Anfrage (z. B. im JSON-Format).
2. MCP-Server empfängt die Anfrage, prüft und leitet sie an das passende Tool weiter.
3. Tool verarbeitet die Anfrage und sendet das Ergebnis an den MCP-Server.
4. MCP-Server gibt das Ergebnis an das LLM zurück, das es in die Antwort integriert.

Vorteile von Tool Calling mit MCP-Servern

- **Modularität:** Einfache Integration neuer Tools und Services.
- **Sicherheit:** Zentralisiertes Management von Zugriffsrechten und Monitoring.
- **Skalierbarkeit:** Parallele Verarbeitung mehrerer Anfragen und Lastverteilung.
- **Nachvollziehbarkeit:** Logging aller Tool-Calls für Audits und Debugging.
- **Domänenspezifische Erweiterbarkeit:** Einbindung von branchenspezifischen Tools und Ontologien.

Wie erkennt ein Agent, dass ein Tool Call nötig ist? (Teil 1)

- **Frage:** Das Modell ist ein Sprachmodell – wie entscheidet es intern, ob es Freitext generiert oder ein Tool aufruft?
- **Trainingszeit: SFT mit Tool-Demonstrations-Daten**
 - Das Modell wird auf Tausenden Beispielen trainiert, in denen korrekte Tool-Aufrufe demonstriert werden.
 - Es lernt assoziative Muster: "Rechenaufgabe → calculator; aktuelles Ereignis → web_search; Datei schreiben → write_file".
 - RLHF bestraft unnötige Tool-Calls (Overhead) und fehlende nötige Calls (falsche Antwort).
- **Inferenzzeit: Tool-Schemata im System-Prompt**
 - Alle verfügbaren Tool-Definitionen (JSON-Schema) werden dem Modell im System-Prompt übergeben.
 - Das Modell führt de facto **Semantic Matching** durch: passt die aktuelle Nutzerfrage zu einer der Tool-Beschreibungen?
 - Effektiv: In-context Few-Shot-Lernen durch die Beschreibungstexte – kein separater Klassifikator.

Wie erkennt ein Agent, dass ein Tool Call nötig ist? (Teil 2)

■ Mechanismus: Strukturierter Token-Output

- Modelle wie GPT-4, Claude 3.5, Gemini sind trainiert, bei Tool-Bedarf spezielle Token-Sequenzen zu erzeugen – ein JSON-Objekt anstelle von Freitext.
- Die Laufzeitumgebung parst die Ausgabe: JSON mit `"name"/"arguments"`? $\Rightarrow$ Tool-Call; sonst $\Rightarrow$ Antworttext.
- Kein explizites `if/else` – die Entscheidung entsteht rein durch die Wahrscheinlichkeitsverteilung des nächsten Tokens.

■ Implizit gelernte Heuristiken:

- **Wissens-Cutoff:** “Was passiert gerade in X?” $\Rightarrow$ Suche nötig.
- **Exakte Berechnung:** Arithmetik, die LLMs unzuverlässig lösen $\Rightarrow$ Calculator-Tool.
- **Datei-/Systemzugriff:** Schreiben, Lesen, Ausführen $\Rightarrow$ Filesystem-/Shell-Tool.
- **Parameterextraktion:** Argumente direkt aus dem Nutzertext befüllen:
“Übersetze ‘Guten Morgen’ ins Englische” $\Rightarrow$ `{"text": "Guten Morgen", "target": "en"}`
- **Parallel Function Calling:** GPT-4/Gemini können mehrere Tool-Calls gleichzeitig erzeugen – reduziert Latenz bei parallelen Teilaufgaben.

Tool Calling: Technische Details – Funktionsdefinitionen (JSON-Schema)

- LLMs nutzen **JSON-Schema**-Beschreibungen, um Tools zu identifizieren und zu parametrisieren.
- **Typische Felder einer Tool-Definition:**
 - **name:** Eindeutiger Bezeichner des Tools.
 - **description:** Natürlichsprachliche Beschreibung, die das LLM interpretiert.
 - **parameters:** JSON-Schema der Eingabeparameter (Typen, Pflichtfelder, Enum-Werte).
- **Entscheidung des Modells:**
 - Das Modell entscheidet anhand der Beschreibung, wann und mit welchen Argumenten ein Tool aufgerufen wird.
 - Ausgabe: strukturierter Tool-Call (JSON-Objekt) statt Freitext.
- **Formate:** OpenAI Function Calling, Anthropic Tool Use, Google Gemini Tools, MCP (Model Context Protocol).

Zukunftsperspektiven für Agentensysteme

- **Verbesserte Autonomie:** Einsatz von LLMs und Ontologien für flexiblere Entscheidungsfindung.
- **Erweiterte Tool- und Ontologie-Nutzung:** Integration von spezialisierten Tools und domänenspezifischen Ontologien.
- **Multi-Agent-Kollaboration:** Entwicklung von Protokollen für effizientere Zusammenarbeit (z. B. JSON, RDF).
- **Domänenspezifische Systeme:** Anpassung an spezifische Branchen wie Medizin, Recht, Industrie.
- **Automatisierte Ontologie-Generierung:** LLMs unterstützen beim Aufbau und der Pflege von Ontologien.

Gedächtnistypen in Agentensystemen

- Agenten nutzen verschiedene Gedächtnisformen, analog zur kognitiven Psychologie:
 - **In-Context-Gedächtnis:** Informationen im aktuellen Prompt-/Kontextfenster.
 - **Episodisches Gedächtnis:** Vergangene Konversationen und Sitzungen, per Vektorsuche abrufbar.
 - **Semantisches Gedächtnis:** Faktenwissen, Nutzerprofile (**MEMORY.md**, Nutzermodellierung).
 - **Prozedurales Gedächtnis:** Abfolgen von Handlungen – **Skills**.
- **Skills** sind wiederverwendbare, strukturierte Anleitungen, die der Agent eigenständig abruft und ausführt.
- **Zusammenspiel:** Episodisches Gedächtnis erinnert was passiert ist; prozedurales Gedächtnis kodiert wie Aufgaben gelöst werden.

Skills: Prozedurales Gedächtnis für Agenten

- **Konzept:** Ein Skill ist eine gespeicherte Schritt-für-Schritt-Anleitung für wiederkehrende Aufgaben – analog zu einer Arbeitsanweisung.
- **Typischer Aufbau einer SKILL.md-Datei:**
 - Titel und Beschreibung des Skills.
 - Trigger-Bedingung: Wann soll der Skill angewendet werden?
 - Geordnete Ausführungsschritte.
 - Benötigte Tools und Ressourcen.
 - Erwartetes Ergebnis und Erfolgskriterien.
- **Hermes Agent³³** als Praxisbeispiel:
 - Erstellt nach komplexen Aufgaben automatisch neue Skills.
 - Verbessert bestehende Skills durch Wiederholung und Feedback.
 - Gemeinschaftlicher Skill-Austausch über <https://agentskills.io/>.

³³<https://github.com/NousResearch/hermes-agent>

Selbstlernender Agenten-Loop (Closed Learning Loop)

■ Ablauf:

1. Agent löst eine komplexe Aufgabe erfolgreich.
2. Agent erkennt, dass die Aufgabe wiederkehrend und kodifizierbar ist.
3. Agent erstellt oder aktualisiert einen Skill (**SKILL.md**).
4. Bei zukünftigen ähnlichen Aufgaben wird der Skill automatisch aktiviert.
5. Schlechte Ergebnisse führen zu Skill-Refinement (Selbstverbesserung).

■ Vorteile:

- Keine manuelle Programmierung von Verhaltensweisen nötig.
- Kontinuierliche Verbesserung durch Erfahrung (lifelong learning).
- Anpassung an spezifischen Nutzer, Kontext und Domäne.

■ Herausforderungen: Qualitätskontrolle, Vermeidung von Skill Drift, Datenschutz, Approval-Gates.

Multi-Agenten-Architekturen

Orchestrator-Subagent:

- Übergeordneter Orchestrator zerlegt Aufgaben und delegiert an Spezialagenten.
- Subagenten berichten zurück; Orchestrator aggregiert Ergebnisse.

Supervisor-Worker:

- Supervisor überwacht Fortschritt und korrigiert Agenten bei Bedarf.
- Robuster gegenüber Fehlern einzelner Agenten.

Peer-to-Peer:

- Direkte Kommunikation ohne zentrale Steuerung.
- Dezentral, resilient, skalierbar.

Frameworks:

- **AutoGen** (Microsoft)
Multi-Agenten-Konversationen.^a
- **CrewAI**
Rollenbasierte Agenten-Teams.^b
- **LangGraph**
Graphbasierte Agenten-Pipelines mit Zustandsverwaltung.^c
- **Hermes Agent**
Selbstlernender Open-Source-Agent mit Skills-System.

^a<https://github.com/microsoft/autogen>

^b<https://github.com/crewAIInc/crewAI>

^c<https://github.com/langchain-ai/langgraph>

Zukunftsvisionen für LLMs

- **Verbesserte Multimodalität:** Integration von Text, Bild, Audio und Video in einem Modell.
- **Domänenspezifische Modelle:** Entwicklung spezialisierter LLMs für Medizin, Recht, Bildung und andere Bereiche.
- **Interaktive KI-Systeme:** Kombination von LLMs mit Robotik und IoT für physische Interaktionen.
- **Selbstlernende Systeme:** Modelle, die sich kontinuierlich durch Interaktion mit der Umgebung verbessern.
- **KI-gestützte Kreativität:** Unterstützung bei Kunst, Musik, Literatur und Design.

Gesellschaftliche Implikationen

- **Arbeitsmarkt:** Automatisierung von Aufgaben und mögliche Auswirkungen auf Beschäftigung.
- **Bildung:** Einsatz von LLMs als personalisierte Lernassistenten.
- **Privatsphäre:** Umgang mit sensiblen Daten und Schutz vor Missbrauch.
- **Regulierung:** Notwendigkeit von Gesetzen und Richtlinien für den verantwortungsvollen Einsatz von KI.
- **Ethik:** Sicherstellung, dass KI-Systeme menschliche Werte respektieren und fördern.

Diskussionspunkte für die Zukunft

- Wie können wir sicherstellen, dass LLMs inklusiv und fair sind?
- Welche Rolle sollten LLMs in der Entscheidungsfindung spielen?
- Wie können wir die Transparenz und Nachvollziehbarkeit von LLMs verbessern?
- Welche Maßnahmen sind notwendig, um Missbrauch zu verhindern?
- Wie können wir die Zusammenarbeit zwischen Mensch und KI optimieren?

KI und Ethik: Herausforderungen und Verantwortung

- **Verantwortung:** Sicherstellung, dass KI-Systeme im Einklang mit ethischen Prinzipien entwickelt und eingesetzt werden.
- **Herausforderungen:**
 - Bias und Diskriminierung in Trainingsdaten und Modellen.
 - Transparenz und Nachvollziehbarkeit von Entscheidungen.
 - Schutz der Privatsphäre und sensible Daten.
 - Verantwortung bei Fehlentscheidungen oder Missbrauch.
- **Gesellschaftliche Auswirkungen:**
 - Einfluss auf Arbeitsplätze und soziale Ungleichheit.
 - Förderung von Inklusion und Diversität.
 - Sicherstellung des Zugangs zu KI-Technologien für alle.

Ethische Prinzipien für KI

- **Fairness:** Vermeidung von Diskriminierung und Bias.
- **Transparenz:** Nachvollziehbarkeit von Entscheidungen und Prozessen.
- **Privatsphäre:** Schutz persönlicher Daten und Minimierung von Überwachung.
- **Sicherheit:** Verhinderung von Missbrauch und Sicherstellung der Robustheit.
- **Verantwortlichkeit:** Klare Zuständigkeiten für die Entwicklung und den Einsatz von KI.

Maßnahmen zur Förderung ethischer KI

- **Regulierung:** Einführung von Gesetzen und Richtlinien für den verantwortungsvollen Einsatz von KI.
- **Audits:** Regelmäßige Überprüfung von Modellen auf Bias und Fairness.
- **Bildung:** Förderung des Bewusstseins für ethische Fragen bei Entwicklern und Nutzern.
- **Interdisziplinäre Zusammenarbeit:** Einbindung von Experten aus Ethik, Recht und Sozialwissenschaften.
- **Open Source:** Transparenz durch Veröffentlichung von Modellen und Trainingsdaten.



Diskussionspunkte zu KI und Ethik

- Wie können wir sicherstellen, dass KI-Systeme fair und inklusiv sind?
- Welche Verantwortung tragen Entwickler und Unternehmen für die Auswirkungen von KI?
- Wie können wir den Missbrauch von KI-Technologien verhindern?
- Welche Rolle sollte die Regulierung bei der Entwicklung und dem Einsatz von KI spielen?
- Wie können wir ethische Prinzipien in den Entwicklungsprozess integrieren?

Zusammenfassung der LLM-Veranstaltung

- **Einführung in NLP und LLMs:** Grundlagen, Herausforderungen und Anwendungen.
- **Sprachdarstellung:** Von One-Hot-Encoding zu modernen Embeddings wie Word2Vec, GloVe und BERT.
- **Transformer-Architektur:** Self-Attention, Encoder-Decoder-Struktur und Vorteile gegenüber RNNs.
- **Fortgeschrittene Modelle:** BERT, GPT, DeepSeek-v3 und ihre spezifischen Stärken.
- **Praktische Anwendungen:** RAG, Agentensysteme und multimodale Verarbeitung.
- **Zukunftsperspektiven:** Multimodalität, domänenspezifische Modelle und ethische Herausforderungen.
- **Diskussion:** Gesellschaftliche Implikationen und verantwortungsvoller Einsatz von LLMs.

LLM Standardwerke

- **Build LLMs from Scratch** (Raschka)
 - <https://github.com/rasbt/LLMs-from-scratch>
 - Praktische Implementierung in PyTorch
- **Transformers for NLP** (Rothman)
 - ISBN 978-1803247335
 - BERT/GPT Anwendungen
- **Deep Learning for NLP** (Goldberg)
 - ISBN 978-3319987305
 - Grundlagen und Anwendungen
- **Natural Language Processing with Transformers** (Tunstall et al.)
 - ISBN 978-1098136789
 - Praxisorientierte Einführung

LLM Forschungsarbeiten

- **Attention Is All You Need** (Vaswani et al., 2017)
 - <https://arxiv.org/abs/1706.03762>
 - Transformer-Architektur
- **BERT Paper** (Devlin et al., 2019)
 - <https://arxiv.org/abs/1810.04805>
 - Bidirektionale Pretraining
- **GPT-3 Paper** (Brown et al., 2020)
 - <https://arxiv.org/abs/2005.14165>
 - Few-Shot Learning
- **GloVe: Global Vectors for Word Representation** (Pennington et al., 2014)
 - <https://aclanthology.org/D14-1162/>
 - Wortvektor-Repräsentationen
- **Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks** (Reimers & Gurevych, 2019)
 - <https://arxiv.org/abs/1908.10084>
 - Satz-Embeddings
- **LLaMA: Open and Efficient Foundation Language Models** (Touvron et al., 2023)
 - <https://arxiv.org/abs/2302.13971>
 - Open-Weight Modelle

LLM Praktische Ressourcen

- **Hugging Face Transformers**
 - <https://github.com/huggingface/transformers>
 - Bibliothek für LLMs
- **LangChain**
 - <https://python.langchain.com/>
 - LLM Orchestrierung
- **LLaMA & LlamaIndex**
 - <https://github.com/facebookresearch/llama>
 - Open-Weight Modelle
- **OpenAI API**
 - <https://platform.openai.com/>
 - Zugriff auf GPT-Modelle
- **Pinecone**
 - <https://www.pinecone.io/>
 - Vektordatenbanken für RAG
- **Weaviate**
 - <https://weaviate.io/>
 - Semantische Suche und Vektorspeicherung